

# NSF\_Funding\_Analysis\_abstract\_only

June 16, 2026

## 1 NSF Funding Analysis (2021–2025)

DS 5500 — Chenjie Gu

This notebook analyzes NSF award data from 2021 to 2025, covering: 1. Funding trends over time by program/CFDA category 2. Top-funded institutions and states per year 3. Program Officer award counts 4. Topic modeling on award abstracts using LDA and BERTopic

**Run cells in order top-to-bottom.** Each section depends on variables defined above it.

All plots are saved as PNG files and then compiled into a single PDF at the end.

### 1.1 1. Setup & Installations

```
[1]: import sys
!{sys.executable} -m pip install -q "urllib3<2" seaborn plotly pandas numpy
↪matplotlib openpyxl geopandas
!{sys.executable} -m pip install -q nltk gensim pyLDAvis
!{sys.executable} -m pip install -q bertopic sentence-transformers umap-learn
↪hdbscan scikit-learn
!{sys.executable} -m pip install -q pypdf Pillow kaleido
!pip install statsmodels
print("All packages installed.")
```

```
Requirement already satisfied: statsmodels in
/Users/_fin.fish_/envs/ds5500/lib/python3.9/site-packages (0.14.6)
Requirement already satisfied: numpy<3,>=1.22.3 in
/Users/_fin.fish_/envs/ds5500/lib/python3.9/site-packages (from statsmodels)
(2.0.2)
Requirement already satisfied: scipy!=1.9.2,>=1.8 in
/Users/_fin.fish_/envs/ds5500/lib/python3.9/site-packages (from statsmodels)
(1.13.1)
Requirement already satisfied: pandas!=2.1.0,>=1.4 in
/Users/_fin.fish_/envs/ds5500/lib/python3.9/site-packages (from statsmodels)
(2.3.3)
Requirement already satisfied: patsy>=0.5.6 in
/Users/_fin.fish_/envs/ds5500/lib/python3.9/site-packages (from statsmodels)
(1.0.2)
Requirement already satisfied: packaging>=21.3 in
```

```
/Users/_fin.fish_/envs/ds5500/lib/python3.9/site-packages (from statsmodels)
(26.2)
Requirement already satisfied: python-dateutil>=2.8.2 in
/Users/_fin.fish_/envs/ds5500/lib/python3.9/site-packages (from
pandas!=2.1.0,>=1.4->statsmodels) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in
/Users/_fin.fish_/envs/ds5500/lib/python3.9/site-packages (from
pandas!=2.1.0,>=1.4->statsmodels) (2026.2)
Requirement already satisfied: tzdata>=2022.7 in
/Users/_fin.fish_/envs/ds5500/lib/python3.9/site-packages (from
pandas!=2.1.0,>=1.4->statsmodels) (2026.2)
Requirement already satisfied: six>=1.5 in
/Users/_fin.fish_/envs/ds5500/lib/python3.9/site-packages (from python-
dateutil>=2.8.2->pandas!=2.1.0,>=1.4->statsmodels) (1.17.0)
All packages installed.
```

```
[2]: import os, re, ast, warnings, logging
      from collections import Counter

      import numpy as np
      import pandas as pd
      import matplotlib
      matplotlib.use('Agg')          # non-interactive backend - avoids display errors
      import matplotlib.pyplot as plt
      import matplotlib.cm as cm
      import matplotlib.colors as mcolors
      import seaborn as sns
      import plotly.express as px
      import plotly.graph_objects as go
      import plotly.colors as pc
      from plotly.subplots import make_subplots

      import nltk
      from nltk.corpus import stopwords
      from nltk.tokenize import word_tokenize
      from nltk.stem import WordNetLemmatizer
      nltk.download('stopwords', quiet=True)
      nltk.download('punkt', quiet=True)
      nltk.download('punkt_tab', quiet=True)
      nltk.download('wordnet', quiet=True)

      import gensim
      from gensim import corpora
      from gensim.models import LdaModel, CoherenceModel
      import pyLDAvis
      import pyLDAvis.gensim_models
```

```

from bertopic import BERTopic
from sentence_transformers import SentenceTransformer
from umap import UMAP
from hdbscan import HDBSCAN
from sklearn.feature_extraction.text import CountVectorizer

warnings.filterwarnings('ignore')
logging.getLogger('gensim').setLevel(logging.ERROR)
logging.getLogger('bertopic').setLevel(logging.ERROR)

plt.rcParams['figure.figsize'] = (12, 6)
sns.set_theme(style='whitegrid')

# Output folder for saved plots
PLOT_DIR = 'plots'
os.makedirs(PLOT_DIR, exist_ok=True)
SAVED_PLOTS = [] # running list - populated by save_fig() below

def save_fig(fig_or_name, name):
    """Save a matplotlib Figure or a Plotly figure to PLOT_DIR and register it.
    ↪ """
    path = os.path.join(PLOT_DIR, f'{name}.png')
    if hasattr(fig_or_name, 'write_image'): # Plotly
        fig_or_name.write_image(path, width=1200, height=600, scale=2)
    elif hasattr(fig_or_name, 'savefig'): # Matplotlib Figure
        fig_or_name.savefig(path, dpi=150, bbox_inches='tight')
        plt.close(fig_or_name)
    SAVED_PLOTS.append(path)
    print(f' Saved → {path}')

print("All imports successful.")
print(f"Plots will be saved to: {os.path.abspath(PLOT_DIR)}")

```

All imports successful.

Plots will be saved to: /Users/\_fin.fish\_/Desktop/NEU/DS5500/plots

## 1.2 2. Load & Inspect Data

```

[3]: df = pd.read_excel('NSF_21_25_SG.xlsx', engine='openpyxl')
print(f'Loaded {len(df):,} rows')
print(df.dtypes.head(20))

```

Loaded 6,477 rows

|              |       |
|--------------|-------|
| Unnamed: 0.2 | int64 |
| Unnamed: 0.1 | int64 |
| Unnamed: 0   | int64 |
| awd_id       | int64 |

```

agcy_id                object
tran_type              object
awd_istr_txt           object
awd_titl_txt           object
cfda_num               object
org_code               int64
po_phone               float64
po_email               object
po_sign_block_name     object
awd_eff_date           datetime64[ns]
awd_exp_date           datetime64[ns]
tot_intn_awd_amt       int64
awd_amount             int64
awd_min_amd_letter_date datetime64[ns]
awd_max_amd_letter_date datetime64[ns]
abstract               object
dtype: object

```

```

[4]: # Data Cleaning

df['awd_eff_date'] = pd.to_datetime(df['awd_eff_date'], errors='coerce')
df['awd_exp_date'] = pd.to_datetime(df['awd_exp_date'], errors='coerce')
df['year'] = df['awd_eff_year'].astype(int)
df['amount'] = pd.to_numeric(df['awd_amount'], errors='coerce')
df['duration_years'] = ((df['awd_exp_date'] - df['awd_eff_date']).dt.days / 365.
    ↳ 25).round(2)
df['institution'] = df['inst.inst_name']
df['state'] = df['inst.inst_state_code']
df['country'] = df['inst.inst_country_name']

# Extract PI name from nested JSON-like string
def extract_pi_name(pi_str):
    try:
        cleaned = str(pi_str).replace('None', '"None"')
        records = ast.literal_eval(cleaned)
        for r in records:
            if r.get('pi_role') == 'Principal Investigator':
                return r.get('pi_full_name', '')
        return records[0].get('pi_full_name', '') if records else ''
    except:
        return ''

# Extract max obligation amount
def extract_oblg(oblg_str):
    try:
        records = ast.literal_eval(str(oblg_str))
        if records:

```

```

        return max(r.get('fund_oblg_amt', 0) for r in records)
    except:
        pass
    return np.nan

df['pi_name']      = df['pi'].apply(extract_pi_name)
df['oblg_amt']     = df['oblg_fy'].apply(extract_oblg)
df['cfda_primary'] = df['cfda_num'].astype(str).str.split(',').str[0].str.
    ↪strip()

# Filter to 2021-2025
df = df[df['year'].between(2021, 2025)].copy()

print(f'Years: {df.year.min()} - {df.year.max()}')
print(f'Filtered to {len(df):,} awards (2021-2025)')
print(f'Missing amounts: {df.amount.isna().sum()}')
print(df.year.value_counts().sort_index())

```

```

Years: 2021 - 2025
Filtered to 6,398 awards (2021-2025)
Missing amounts: 0
year
2021    1278
2022    1441
2023    1462
2024    1194
2025    1023
Name: count, dtype: int64

```

### 1.3 3. Funding Trends Over Time

```

[5]: # Total NSF funding per year
yearly = df.groupby('year')['amount'].sum().reset_index()
yearly.columns = ['Year', 'Total Funding ($)']

fig = px.bar(
    yearly, x='Year', y='Total Funding ($)',
    title='Total NSF Funding per Year (2021-2025)',
    text_auto='.2s'
)
fig.update_traces(marker_color='steelblue')
fig.update_layout(showlegend=False)
save_fig(fig, '01_total_funding_per_year')
fig.show()

```

Saved → plots/01\_total\_funding\_per\_year.png

```
[6]: # Funding by CFDA category
df_cfda = df.copy()
df_cfda['cfda_split'] = df_cfda['cfda_num'].astype(str).str.split(',')
df_cfda = df_cfda.explode('cfda_split')
df_cfda['cfda_split'] = df_cfda['cfda_split'].str.strip()
df_cfda['cfda_split'] = df_cfda['cfda_split'].replace('47.070', '47.07')

cfda_names = {
    '47.07': 'Computer & Info Science',
    '47.049': 'Math & Physical Sciences',
    '47.050': 'Geosciences',
    '47.074': 'Biological Sciences',
    '47.075': 'Social Sciences',
    '47.076': 'Education',
    '47.079': 'STEM Education',
    '47.083': 'Office of Integrative Activities',
    '47.041': 'Engineering',
}
df_cfda['cfda_label'] = df_cfda['cfda_split'].map(cfda_names).
↳ fillna(df_cfda['cfda_split'])

cfda_year = df_cfda.groupby(['year', 'cfda_label'])['amount'].sum().
↳ reset_index()
top_cfda = (
    cfda_year.groupby('cfda_label')['amount']
    .sum().nlargest(6).index.tolist()
)

fig = px.line(
    cfda_year[cfda_year['cfda_label'] == 'Computer & Info Science'],
    x='year', y='amount', color='cfda_label',
    title='Computer & Info Science Funding Over Time',
    labels={'amount': 'Total Funding ($)', 'year': 'Year', 'cfda_label': 'CFDA_
↳ Category'},
    markers=True
)
save_fig(fig, '02_cfda_funding_trends')
fig.show()
```

Saved → plots/02\_cfda\_funding\_trends.png

```
[7]: # Treemap: total funding by CFDA
cfda_total = cfda_year.groupby('cfda_label')['amount'].sum().reset_index()

fig = px.treemap(
    cfda_total, path=['cfda_label'], values='amount',
    title='Total NSF Funding by CFDA Category (2021-2025)',
```

```

        color='amount', color_continuous_scale='Blues',
        labels={'amount': 'Total Funding ($)'}
    )
    fig.update_traces(textinfo='label+value+percent root')
    save_fig(fig, '04_cfda_treemap')
    fig.show()

```

Saved → plots/04\_cfda\_treemap.png

```

[8]: # CSE division funding per year
print('Division value counts:')
print(df['div_abbr'].value_counts())

div_yearly = df.groupby(['year', 'div_abbr']).agg(
    total_funding=('amount', 'sum'),
    num_awards=('awd_id', 'count')
).reset_index()

div_names = {
    'CSE': 'Computer and Information Science and Engineering (CISE)',
    'OAC': 'Advanced Cyberinfrastructure (CISE/OAC)',
    'CNS': 'Computer and Network Systems (CISE/CNS)',
    'CCF': 'Computing and Communication Foundations (CISE/CCF)',
    'IIS': 'Information and Intelligent Systems (CISE/IIS)',
}

div_yearly['division'] = div_yearly['div_abbr'].map(div_names).
    ↪ fillna(div_yearly['div_abbr'])

fig1 = px.line(
    div_yearly, x='year', y='total_funding', color='division',
    markers=True,
    title='NSF CSE Funding by Division per Year',
    labels={'total_funding': 'Total Funding ($)', 'year': 'Year', 'division': 'Division'}
)
fig1.update_xaxes(tickvals=[2021, 2022, 2023, 2024, 2025])
save_fig(fig1, '05_div_funding_per_year')
fig1.show()

fig2 = px.line(
    div_yearly, x='year', y='num_awards', color='division',
    markers=True,
    title='NSF CSE Award Count by Division per Year',
    labels={'num_awards': 'Number of Awards', 'year': 'Year', 'division': 'Division'}
)
fig2.update_xaxes(tickvals=[2021, 2022, 2023, 2024, 2025])

```

```
save_fig(fig2, '06_div_awards_per_year')
fig2.show()
```

Division value counts:

```
div_abbr
CNS      2166
IIS      1601
CCF      1445
OAC      1186
```

Name: count, dtype: int64

Saved → plots/05\_div\_funding\_per\_year.png

Saved → plots/06\_div\_awards\_per\_year.png

```
[9]: # Division totals (all years combined)
div_total = (
    df.groupby('div_abbr').agg(
        total_funding=('amount', 'sum'),
        num_awards=('awd_id', 'count')
    ).reset_index()
    .sort_values('total_funding', ascending=False)
)
div_names2 = {
    'OAC': 'Advanced Cyberinfrastructure (OAC)',
    'CNS': 'Computer and Network Systems (CNS)',
    'CCF': 'Computing and Communication Foundations (CCF)',
    'IIS': 'Information and Intelligent Systems (IIS)',
}
div_total['division'] = div_total['div_abbr'].map(div_names2).
    ↪fillna(div_total['div_abbr'])

fig1 = px.bar(
    div_total, x='division', y='total_funding',
    title='NSF CSE Total Funding by Division (2021-2025)',
    labels={'total_funding': 'Total Funding ($)', 'division': 'Division'},
    height=500
)
fig1.update_traces(marker_color='steelblue')
fig1.update_layout(xaxis_tickangle=-15)
save_fig(fig1, '07_div_total_funding')
fig1.show()

fig2 = px.bar(
    div_total, x='division', y='num_awards',
    title='NSF CSE Total Awards by Division (2021-2025)',
    labels={'num_awards': 'Number of Awards', 'division': 'Division'},
    height=500
)
```



```
fig2.update_traces(marker_color='steelblue')
fig2.update_layout(xaxis_tickangle=-15)
save_fig(fig2, '08_div_total_awards')
fig2.show()
```

Saved → plots/07\_div\_total\_funding.png

Saved → plots/08\_div\_total\_awards.png

```
[10]: print(df.columns.tolist())
```

```
print('pgm_ele sample:')
print(df['pgm_ele'].value_counts().head(20))
print('\npgm_ref sample:')
print(df['pgm_ref'].value_counts().head(20))
```

```
['Unnamed: 0.2', 'Unnamed: 0.1', 'Unnamed: 0', 'awd_id', 'agcy_id', 'tran_type',
'awd_istr_txt', 'awd_titl_txt', 'cfda_num', 'org_code', 'po_phone', 'po_email',
'po_sign_block_name', 'awd_eff_date', 'awd_exp_date', 'tot_intn_awd_amt',
'awd_amount', 'awd_min_amd_letter_date', 'awd_max_amd_letter_date', 'abstract',
'awd_arra_amount', 'dir_abbr', 'org_dir_long_name', 'div_abbr',
'org_div_long_name', 'awd_agcy_code', 'fund_agcy_code', 'pi', 'pgm_ele',
'pgm_ref', 'app_fund', 'oblg_fy', 'por', 'inst.inst_name',
'inst.inst_street_address', 'inst.inst_street_address_2', 'inst.inst_city_name',
'inst.inst_state_code', 'inst.inst_state_name', 'inst.inst_phone_num',
'inst.inst_zip_code', 'inst.inst_country_name', 'inst.cong_dist_code',
'inst.st_cong_dist_code', 'inst.org_lgl_bus_name', 'inst.org_prnt_uei_num',
'inst.org_uei_num', 'perf_inst.perf_inst_name', 'perf_inst.perf_str_addr',
'perf_inst.perf_city_name', 'perf_inst.perf_st_code', 'perf_inst.perf_st_name',
'perf_inst.perf_zip_code', 'perf_inst.perf_ctry_code',
'perf_inst.perf_cong_dist', 'perf_inst.perf_st_cong_dist',
'perf_inst.perf_ctry_name', 'perf_inst.perf_ctry_flag', 'filename', 'corpus',
'por.por_cntn', 'por.por_txt_cntn', 'awd_istr_code', 'org_code_dir',
'org_code_div', 'po_phon', 'po_email', 'sign_blk_name', 'sign_blk_name2',
'awd_eff_date_epoch', 'awd_exp_date_epoch', 'amd_amt', 'last_prop_id',
'amd_ltr_date', 'amd_ltr_date_epoch', 'amd_ltr_date2', 'amd_ltr_date2_epoch',
'abst_narr_txt', 'amd_amt2', 'awd_amt_cat', 'nsf_par_use_flag', 'org_long_name',
'org_long_name2', 'is_hist_awd', 'inst.inst_id', 'inst.inst_shrt_name',
'inst.str_addr', 'inst.str_addr_2', 'inst.cty_name', 'inst.st_code',
'inst.st_name', 'inst.inst_phon_num', 'inst.zip_code', 'inst.ctry_code',
'inst.ctry_name', 'inst.cnty_name', 'inst.ctry_flag', 'perf_inst.perf_inst_id',
'perf_inst.perf_cnty_name', 'awd_eff_year', 'number_of_words', 'year', 'amount',
'duration_years', 'institution', 'state', 'country', 'pi_name', 'oblg_amt',
'cfda_primary']
pgm_ele sample:
pgm_ele
[{'pgm_ele_code': '806000', 'pgm_ele_name': 'Secure & Trustworthy Cyberspace'}]
532
[{'pgm_ele_code': '779800', 'pgm_ele_name': 'Software & Hardware Foundation'}]
```

```

359
[{'pgm_ele_code': '736400', 'pgm_ele_name': 'Info Integration & Informatics'}]
357
[{'pgm_ele_code': '736700', 'pgm_ele_name': 'HCC-Human-Centered Computing'}]
334
[{'pgm_ele_code': '749500', 'pgm_ele_name': 'Robust Intelligence'}]
301
[{'pgm_ele_code': '779700', 'pgm_ele_name': 'Comm & Information Foundations'}]
195
[{'pgm_ele_code': '033Y00', 'pgm_ele_name': 'S&CC: Smart & Connected Commun'}]
190
[{'pgm_ele_code': '736300', 'pgm_ele_name': 'Networking Technology and Syst'}]
180
[{'pgm_ele_code': '779600', 'pgm_ele_name': 'Algorithmic Foundations'}]
174
[{'pgm_ele_code': '735400', 'pgm_ele_name': 'CSR-Computer Systems Research'}]
174
[{'pgm_ele_code': '800400', 'pgm_ele_name': 'Software Institutes'}]
160
[{'pgm_ele_code': '735900', 'pgm_ele_name': 'CCRI-CISE Cmnty Rsrch Infrstrc'}]
154
[{'pgm_ele_code': '113900', 'pgm_ele_name': 'RSCH EXPER FOR UNDERGRAD SITES'}]
140
[{'pgm_ele_code': '044Y00', 'pgm_ele_name': 'CyberTraining - Training-based'}]
139
[{'pgm_ele_code': '173Y00', 'pgm_ele_name': 'CISE MSI Research Expansion'}]
138
[{'pgm_ele_code': '791800', 'pgm_ele_name': 'CPS-Cyber-Physical Systems'}]
134
[{'pgm_ele_code': '801800', 'pgm_ele_name': 'Smart and Connected Health'}]
109
[{'pgm_ele_code': '808000', 'pgm_ele_name': 'Campus Cyberinfrastructure'}]
103
[{'pgm_ele_code': '094Y00', 'pgm_ele_name': 'FMitF: Formal Methods in the F'}]
98
[{'pgm_ele_code': '090Y00', 'pgm_ele_name': 'OAC-Advanced Cyberinfrast Core'}]
89
Name: count, dtype: int64

```

pgm\_ref sample:

```

pgm_ref
[{'pgm_ref_code': '042Z', 'pgm_ref_txt': 'S&CC: Smart and Connected
Communities'}]
167
[{'pgm_ref_code': '7923', 'pgm_ref_txt': 'SMALL PROJECT'}]
161
[{'pgm_ref_code': '025Z', 'pgm_ref_txt': 'SaTC: Secure and Trustworthy
Cyberspace'}, {'pgm_ref_code': '7923', 'pgm_ref_txt': 'SMALL PROJECT'}]

```

```

98
[{'pgm_ref_code': '7367', 'pgm_ref_txt': 'Cyber-Human Systems'},
{'pgm_ref_code': '7924', 'pgm_ref_txt': 'MEDIUM PROJECT'}]
96
[{'pgm_ref_code': '7495', 'pgm_ref_txt': 'ROBUST INTELLIGENCE'},
{'pgm_ref_code': '7923', 'pgm_ref_txt': 'SMALL PROJECT'}]
90
[{'pgm_ref_code': '7924', 'pgm_ref_txt': 'MEDIUM PROJECT'}]
89
[{'pgm_ref_code': '7359', 'pgm_ref_txt': 'COMPUTING RES INFRASTRUCTURE'}]
86
[{'pgm_ref_code': '7364', 'pgm_ref_txt': 'INFO INTEGRATION & INFORMATICS'},
{'pgm_ref_code': '7923', 'pgm_ref_txt': 'SMALL PROJECT'}]
80
[{'pgm_ref_code': '025Z', 'pgm_ref_txt': 'SaTC: Secure and Trustworthy
Cyberspace'}, {'pgm_ref_code': '7924', 'pgm_ref_txt': 'MEDIUM PROJECT'}]
73
[{'pgm_ref_code': '7916', 'pgm_ref_txt': 'EAGER'}]
73
[{'pgm_ref_code': '7367', 'pgm_ref_txt': 'Cyber-Human Systems'},
{'pgm_ref_code': '7923', 'pgm_ref_txt': 'SMALL PROJECT'}]
73
[{'pgm_ref_code': '9150', 'pgm_ref_txt': 'EXP PROG TO STIM COMP RES'}]
71
[{'pgm_ref_code': '9250', 'pgm_ref_txt': 'REU SITE-Res Exp for Ugrd Site'}]
67
[{'pgm_ref_code': '025Z', 'pgm_ref_txt': 'SaTC: Secure and Trustworthy
Cyberspace'}, {'pgm_ref_code': '7556', 'pgm_ref_txt': 'CONFERENCE AND
WORKSHOPS'}]
64
[{'pgm_ref_code': '7495', 'pgm_ref_txt': 'ROBUST INTELLIGENCE'},
{'pgm_ref_code': '7924', 'pgm_ref_txt': 'MEDIUM PROJECT'}]
64
[{'pgm_ref_code': '7364', 'pgm_ref_txt': 'INFO INTEGRATION & INFORMATICS'},
{'pgm_ref_code': '7924', 'pgm_ref_txt': 'MEDIUM PROJECT'}]
63
[{'pgm_ref_code': '122Z', 'pgm_ref_txt': 'CI Training and Professionals'},
{'pgm_ref_code': '7361', 'pgm_ref_txt': 'EDUCATION AND WORKFORCE'},
{'pgm_ref_code': '7569', 'pgm_ref_txt': 'CYBERINFRASTRUCTURE/SCIENCE'}]
60
[{'pgm_ref_code': '8018', 'pgm_ref_txt': 'Smart and Connected Health'}]
59
[{'pgm_ref_code': '7367', 'pgm_ref_txt': 'Cyber-Human Systems'},
{'pgm_ref_code': '7556', 'pgm_ref_txt': 'CONFERENCE AND WORKSHOPS'}]
55
[{'pgm_ref_code': '071Z', 'pgm_ref_txt': 'FMitF-Formal Methods in the Field'}]
53
Name: count, dtype: int64

```

```

[11]: import ast

# Parse pgm_ele into usable program names
def extract_pgm_names(val):
    try:
        parsed = ast.literal_eval(val)
        return [p['pgm_ele_name'] for p in parsed]
    except:
        return []

df_pgm = df.copy()
df_pgm['pgm_names'] = df_pgm['pgm_ele'].apply(extract_pgm_names)
df_pgm = df_pgm.explode('pgm_names')
df_pgm = df_pgm[df_pgm['pgm_names'].notna() & (df_pgm['pgm_names'] != '')]

# Funding by division and program
div_pgm = (
    df_pgm.groupby(['div_abbr', 'pgm_names']).agg(
        total_funding=('amount', 'sum'),
        num_awards=('awd_id', 'count')
    ).reset_index()
)

div_names = {
    'OAC': 'Advanced Cyberinfrastructure (OAC)',
    'CNS': 'Computer and Network Systems (CNS)',
    'CCF': 'Computing and Communication Foundations (CCF)',
    'IIS': 'Information and Intelligent Systems (IIS)',
}

div_pgm['division'] = div_pgm['div_abbr'].map(div_names).
    ↪fillna(div_pgm['div_abbr'])

top_pgms = (
    div_pgm.groupby(['division', 'pgm_names'])['total_funding']
        .sum()
        .reset_index()
        .sort_values(['division', 'total_funding'], ascending=[True, False])
)

div_pgm_top = div_pgm[div_pgm['pgm_names'].isin(top_pgms['pgm_names'])]

# Funding distribution by Division and Program - individual charts
for metric, ylabel, slug in [
    ('total_funding', 'Total Funding ($)', 'funding'),
    ('num_awards', 'Number of Awards', 'awards'),
]:
    for div_abbr, div_name in div_names.items():

```

```

        div_data = div_pgm_top[div_pgm_top['div_abbr'] == div_abbr].
↪sort_values(metric, ascending=False)

        if div_data.empty:
            continue

        fig = px.bar(
            div_data,
            x='pgm_names', y=metric,
            title=f'{div_name} - {ylabel} by Program (2021-2025)',
            labels={metric: ylabel, 'pgm_names': 'Program'},
            height=550,
        )
        fig.update_traces(marker_color=pc.qualitative.Light24[list(div_names.
↪keys()).index(div_abbr)])
        fig.update_layout(xaxis_tickangle=-45, xaxis_tickfont_size=9)
        save_fig(fig, f'33_div_pgm_{div_abbr.lower()}_{slug}')
        fig.show()

```

Saved → plots/33\_div\_pgm\_oac\_funding.png

Saved → plots/33\_div\_pgm\_cns\_funding.png

Saved → plots/33\_div\_pgm\_ccf\_funding.png

Saved → plots/33\_div\_pgm\_iis\_funding.png

Saved → plots/33\_div\_pgm\_oac\_awards.png

Saved → plots/33\_div\_pgm\_cns\_awards.png

Saved → plots/33\_div\_pgm\_ccf\_awards.png

Saved → plots/33\_div\_pgm\_iis\_awards.png

## 1.4 4. Institutional & Geographic Analysis

```

[12]: import numpy as np
import plotly.graph_objects as go

# grand totals (denominators)
grand_funding = df['amount'].sum()
grand_awards = df['awd_id'].count()

TOP_N = 100
TOP_BUBBLE = 50

# Institution totals (all schools, used for treemap + ranking)
inst_all = (
    df.groupby('institution').agg(
        total_funding=('amount', 'sum'),

```

```

        num_awards=('awd_id', 'count')
    ).reset_index()
)
inst_all = inst_all[inst_all['total_funding'] > 0].reset_index(drop=True)

# Bar: top 100 schools by funding
inst_total = (
    inst_all.sort_values('total_funding', ascending=False)
    .head(TOP_N).reset_index(drop=True)
)

bar_fund_share = inst_total['total_funding'].sum() / grand_funding * 100
print(f'Top {TOP_N} schools by funding account for {bar_fund_share:.1f}% of
↳total funding.')

fig1 = px.bar(
    inst_total, x='institution', y='total_funding',
    title=(f'Total NSF CSE Funding by Institution - Top {TOP_N} (2021-2025)'
           f'<br><sup>These schools = {bar_fund_share:.1f}% of funding</sup>'),
    labels={'total_funding': 'Total Funding ($)', 'institution': 'Institution'},
    height=600
)
fig1.update_traces(marker_color='steelblue')
fig1.update_layout(xaxis_tickangle=-90, xaxis_tickfont_size=7)
save_fig(fig1, '10_inst_total_bar')
fig1.show()

# Treemap: ALL schools
fig2 = px.treemap(
    inst_all, path=['institution'], values='total_funding',
    title='NSF CSE Funding by Institution - Treemap, All Schools (2021-2025)',
    color='total_funding', color_continuous_scale='Blues',
)
fig2.update_traces(textinfo='label+value')
save_fig(fig2, '11_inst_treemap')
fig2.show()

# Bubble: top 50 schools by funding
inst_counts = (
    inst_all.sort_values('total_funding', ascending=False)
    .head(TOP_BUBBLE).reset_index(drop=True)
)

bub_fund_share = inst_counts['total_funding'].sum() / grand_funding * 100
bub_award_share = inst_counts['num_awards'].sum() / grand_awards * 100
print(f'Top {TOP_BUBBLE} schools by funding account for {bub_fund_share:.1f}%
↳of funding ')

```

```

        f'and {bub_award_share:.1f}% of awards.')

fig3 = px.scatter(
    inst_counts, x='num_awards', y='total_funding',
    size='total_funding', hover_name='institution',
    text='institution',
    title=(f'NSF CSE Institutions - Bubble Chart (Top {TOP_BUBBLE}, 2021-2025)'
           f'<br><sup>These schools = {bub_fund_share:.1f}% of funding, '
           f'{bub_award_share:.1f}% of awards</sup>'),
    labels={'total_funding': 'Total Funding ($)', 'num_awards': 'Number of_
↳ Awards'},
    color='total_funding', color_continuous_scale='Teal', height=600
)
fig3.update_traces(textposition='top center', textfont_size=7)
save_fig(fig3, '12_inst_bubble')
fig3.show()

```

Top 100 schools by funding account for 76.7% of total funding.

Saved → plots/10\_inst\_total\_bar.png

Saved → plots/11\_inst\_treemap.png

Top 50 schools by funding account for 56.6% of funding and 50.3% of awards.

Saved → plots/12\_inst\_bubble.png

```

[13]: # AWARD-COUNT VERSIONS

# Bar: top 100 schools by award count
inst_total_aw = (
    inst_all.sort_values('num_awards', ascending=False)
    .head(TOP_N).reset_index(drop=True)
)

bar_award_share = inst_total_aw['num_awards'].sum() / grand_awards * 100
print(f'Top {TOP_N} schools by awards account for {bar_award_share:.1f}% of all_
↳ awards.')

fig4 = px.bar(
    inst_total_aw, x='institution', y='num_awards',
    title=(f'Total NSF CSE Awards by Institution - Top {TOP_N} (2021-2025)'
           f'<br><sup>These schools = {bar_award_share:.1f}% of awards</sup>'),
    labels={'num_awards': 'Number of Awards', 'institution': 'Institution'},
    height=600
)
fig4.update_traces(marker_color='steelblue')
fig4.update_layout(xaxis_tickangle=-90, xaxis_tickfont_size=7)
save_fig(fig4, '13_inst_total_bar_awards')
fig4.show()

```

```

# Treemap: ALL schools, sized by award count
fig5 = px.treemap(
    inst_all, path=['institution'], values='num_awards',
    title='NSF CSE Awards by Institution - Treemap, All Schools (2021-2025)',
    color='num_awards', color_continuous_scale='Blues',
)
fig5.update_traces(textinfo='label+value')
save_fig(fig5, '14_inst_treemap_awards')
fig5.show()

# Bubble: top 50 schools by award count
inst_counts_aw = (
    inst_all.sort_values('num_awards', ascending=False)
    .head(TOP_BUBBLE).reset_index(drop=True)
)

bub_aw_award_share = inst_counts_aw['num_awards'].sum() / grand_awards * 100
bub_aw_fund_share = inst_counts_aw['total_funding'].sum() / grand_funding * 100
print(f'Top {TOP_BUBBLE} schools by awards account for {bub_aw_award_share:.1f}% of awards '
      f'and {bub_aw_fund_share:.1f}% of funding.')

fig6 = px.scatter(
    inst_counts_aw, x='num_awards', y='total_funding',
    size='num_awards', hover_name='institution',
    text='institution',
    title=(f'NSF CSE Institutions - Bubble Chart by Awards (Top {TOP_BUBBLE}, 2021-2025)'
          f'<br><sup>These schools = {bub_aw_award_share:.1f}% of awards, '
          f'{bub_aw_fund_share:.1f}% of funding</sup>'),
    labels={'total_funding': 'Total Funding ($)', 'num_awards': 'Number of Awards'},
    color='num_awards', color_continuous_scale='Teal', height=600
)
fig6.update_traces(textposition='top center', textfont_size=7)
save_fig(fig6, '15_inst_bubble_awards')
fig6.show()

```

Top 100 schools by awards account for 71.9% of all awards.

Saved → plots/13\_inst\_total\_bar\_awards.png

Saved → plots/14\_inst\_treemap\_awards.png

Top 50 schools by awards account for 50.6% of awards and 56.0% of funding.

Saved → plots/15\_inst\_bubble\_awards.png



```
[14]: # State funding maps with shrunken AK/HI insets + TRUE campus locations
import geopandas as gpd
import shapely.affinity as aff

# Embedded campus coordinates (lat, lon), keyed by substrings in names
CAMPUS = {
    'carnegie mellon': (40.4433,-79.9436), 'georgia tech': (33.7756,-84.3963),
    'illinois at urbana': (40.1020,-88.2272), 'arizona state': (33.4242,-111.9281),
    'michigan - ann arbor': (42.2780,-83.7382), 'purdue': (40.4237,-86.9212),
    'california-san diego': (32.8801,-117.2340), 'washington': (47.6553,-122.3035),
    'cornell': (42.4534,-76.4735), 'southern california': (34.0224,-118.2851),
    'texas at austin': (30.2849,-97.7341), 'massachusetts institute': (42.3601,-71.
    ↪0942),
    'stanford': (37.4275,-122.1697), 'berkeley': (37.8719,-122.2585),
    'wisconsin-madison': (43.0766,-89.4125), 'maryland, college park': (38.
    ↪9869,-76.9426),
    'maryland-college park': (38.9869,-76.9426), 'pennsylvania state': (40.
    ↪7982,-77.8599),
    'rutgers': (40.5008,-74.4474), 'northeastern': (42.3398,-71.0892),
    'new york university': (40.7295,-73.9965), 'virginia polytechnic': (37.
    ↪2284,-80.4234),
    'virginia tech': (37.2284,-80.4234), 'chicago': (41.7886,-87.5987),
    'north carolina state': (35.7847,-78.6821), 'utah': (40.7649,-111.8421),
    'minnesota': (44.9740,-93.2277), 'massachusetts amherst': (42.3868,-72.5301),
    'northwestern': (42.0565,-87.6753), 'virginia main': (38.0336,-78.5080),
    'california-los angeles': (34.0689,-118.4452), 'princeton': (40.3431,-74.6551),
    'columbia': (40.8075,-73.9626), 'harvard': (42.3770,-71.1167),
    'yale': (41.3163,-72.9223), 'duke': (36.0014,-78.9382),
    'ohio state': (40.0067,-83.0305), 'texas a&m': (30.6188,-96.3365),
    'boston university': (42.3505,-71.1054), 'brown': (41.8268,-71.4025),
    'california-davis': (38.5382,-121.7617), 'california-irvine': (33.6405,-117.
    ↪8443),
    'california-santa barbara': (34.4140,-119.8489), 'california-riverside': (33.
    ↪9737,-117.3281),
    'colorado': (40.0076,-105.2659), 'florida': (29.6436,-82.3549),
    'indiana': (39.1682,-86.5230), 'iowa state': (42.0267,-93.6465),
    'johns hopkins': (39.3299,-76.6205), 'michigan state': (42.7018,-84.4822),
    'oregon': (44.0448,-123.0726), 'pittsburgh': (40.4444,-79.9608),
    'rochester': (43.1287,-77.6277), 'rice': (29.7174,-95.4018),
    'south carolina': (33.9938,-81.0301), 'tennessee': (35.9544,-83.9295),
    'vanderbilt': (36.1447,-86.8027), 'washington university': (38.6488,-90.3108),
    'buffalo': (43.0008,-78.7890), 'stony brook': (40.9123,-73.1234),
    'george mason': (38.8315,-77.3110), 'notre dame': (41.7052,-86.2353),
    'syracuse': (43.0392,-76.1351), 'connecticut': (41.8077,-72.2540),
    'delaware': (39.6780,-75.7506), 'houston': (29.7199,-95.3422),
    'kansas': (38.9543,-95.2558), 'kentucky': (38.0307,-84.5040),
    'louisiana state': (30.4133,-91.1800),

```

```

    'alaska': (64.8576, -147.8260), 'hawaii': (21.2969, -157.8171),
}
CAMPUS_KEYS = sorted(CAMPUS, key=len, reverse=True)

def campus_coords(name):
    low = str(name).lower()
    for k in CAMPUS_KEYS:
        if k in low:
            return CAMPUS[k]
    return (None, None)

# Institution funding + coordinate resolution
inst_state = df.groupby(['institution', 'state'])['amount'].sum().reset_index()
cc = inst_state['institution'].apply(campus_coords)
inst_state['lat'] = [c[0] for c in cc]
inst_state['lon'] = [c[1] for c in cc]

matched = inst_state['lat'].notna().sum()
print(f'Matched {matched}/{len(inst_state)} institutions to real campus coords '
      f'({matched/len(inst_state)*100:.0f}%%); rest use state centroid.')

# State geometry + centroid fallback
gdf = gpd.read_file(
    'https://raw.githubusercontent.com/nvkelso/natural-earth-vector/master/'
    'geojson/ne_110m_admin_1_states_provinces.geojson'
)
gdf = gdf[gdf['iso_a2'] == 'US'].copy()
gdf['state'] = gdf['postal'].str.strip()

gdf_proj = gdf.to_crs('EPSG:5070')
cent = gdf_proj.geometry.centroid.to_crs('EPSG:4326')
state_centroid = dict(zip(gdf['state'], zip(cent.x, cent.y))) # (lon, lat)

def resolve(r):
    if pd.notna(r['lat']):
        return r['lat'], r['lon']
    c = state_centroid.get(r['state'])
    return (c[1], c[0]) if c else (None, None)

res = inst_state.apply(resolve, axis=1, result_type='expand')
inst_state['plat'], inst_state['plon'] = res[0], res[1]

# State funding per year

```

```

state_year = df.groupby(['year', 'state'])['amount'].sum().reset_index()
vmin, vmax = state_year['amount'].min(), state_year['amount'].max()
years = sorted(state_year['year'].unique())

CONUS = gdf[~gdf['state'].isin(['AK', 'HI'])]
minx, miny, maxx, maxy = CONUS.total_bounds

def inset_params(geom, scale, target_left, target_bottom):
    """Return (scale, cx, cy, dx, dy) describing the scale-about-centroid +
    translate that place_inset applies, so points can be moved the same way."""
    cx, cy = geom.centroid.x, geom.centroid.y
    scaled = aff.scale(geom, xfact=scale, yfact=scale, origin='centroid')
    b = scaled.bounds
    return scale, cx, cy, target_left - b[0], target_bottom - b[1]

def place_inset_geom(geom, params):
    scale, cx, cy, dx, dy = params
    scaled = geom.apply(lambda g: aff.scale(g, xfact=scale, yfact=scale,
    ↪origin='centroid'))
    return gpd.GeoSeries(
        [aff.translate(g, xoff=dx, yoff=dy) for g in scaled],
        index=geom.index, crs=geom.crs
    )

def transform_pts(lon, lat, params):
    """Apply the same affine to point coords (vectorized)."""
    scale, cx, cy, dx, dy = params
    return cx + (lon - cx) * scale + dx, cy + (lat - cy) * scale + dy

AK_SCALE, HI_SCALE = 0.40, 1.0

for yr in years:
    yr_data = state_year[state_year['year'] == yr]
    merged = gdf.merge(yr_data, on='state', how='left')

    fig_map, ax = plt.subplots(1, 1, figsize=(14, 8))

    conus = merged[~merged['state'].isin(['AK', 'HI'])]
    conus.plot(column='amount', ax=ax, cmap='YlOrRd', vmin=vmin, vmax=vmax,
                edgecolor='black', linewidth=0.3, legend=True,
                legend_kwds={'label': 'Total Funding ($)', 'orientation':
    ↪'vertical',

```

```

        'shrink': 0.6, 'format': lambda x, _: f'${x/1e6:.
↪Of}M'},
        missing_kwds={'color': 'lightgrey'})

smax = inst_state['amount'].max()

def draw_points(sub, ax):
    if len(sub):
        sizes = (sub['amount'] / smax * 250) + 4
        ax.scatter(sub['px'], sub['py'], s=sizes, color='navy', alpha=0.45,
                    edgecolor='white', linewidth=0.3, zorder=5)

# CONUS polygons already drawn; CONUS points
conus_pts = inst_state[~inst_state['state'].isin(['AK', 'HI'])].copy()
conus_pts['px'], conus_pts['py'] = conus_pts['plon'], conus_pts['plat']
conus_pts = conus_pts[conus_pts['px'].between(minx - 2, maxx + 2) &
                      conus_pts['py'].between(miny - 2, maxy + 2)]
draw_points(conus_pts, ax)

# Alaska polygon + points
ak = merged[merged['state'] == 'AK']
if not ak.empty:
    ak = ak.copy()
    ak_params = inset_params(ak.geometry.iloc[0], AK_SCALE,
                             target_left=minx - 3, target_bottom=miny - 10)
    ak['geometry'] = place_inset_geom(ak.geometry, ak_params)
    ak.plot(column='amount', ax=ax, cmap='YlOrRd', vmin=vmin, vmax=vmax,
            edgecolor='black', linewidth=0.3, missing_kwds={'color': '
↪lightgrey'})
    ak_pts = inst_state[inst_state['state'] == 'AK'].copy()
    if len(ak_pts):
        ak_pts['px'], ak_pts['py'] = transform_pts(ak_pts['plon'],
↪ak_pts['plat'], ak_params)
        draw_points(ak_pts, ax)

# Hawaii polygon + points
hi = merged[merged['state'] == 'HI']
if not hi.empty:
    hi = hi.copy()
    hi_params = inset_params(hi.geometry.iloc[0], HI_SCALE,
                             target_left=minx + 16, target_bottom=miny - 8)
    hi['geometry'] = place_inset_geom(hi.geometry, hi_params)
    hi.plot(column='amount', ax=ax, cmap='YlOrRd', vmin=vmin, vmax=vmax,
            edgecolor='black', linewidth=0.3, missing_kwds={'color': '
↪lightgrey'})
    hi_pts = inst_state[inst_state['state'] == 'HI'].copy()
    if len(hi_pts):

```

```

        hi_pts['px'], hi_pts['py'] = transform_pts(hi_pts['plon'],
↳hi_pts['plat'], hi_params)
        draw_points(hi_pts, ax)

    ax.set_xlim(minx - 3, maxx + 2)
    ax.set_ylim(miny - 12, maxy + 2)
    ax.set_title(f'NSF CSE Funding by State - {yr}', fontsize=16,
↳fontweight='bold')
    ax.axis('off')
    plt.tight_layout()
    save_fig(fig_map, f'13_state_map_{yr}')
    plt.show()

```

Matched 144/568 institutions to real campus coords (25%); rest use state centroid.

```

Saved → plots/13_state_map_2021.png
Saved → plots/13_state_map_2022.png
Saved → plots/13_state_map_2023.png
Saved → plots/13_state_map_2024.png
Saved → plots/13_state_map_2025.png

```

```

[15]: # Diagnostic: why are points falling back to centroids?

# 1. Did the coordinate dataframe load, and what's in it?
print('=== uni dataframe ===')
try:
    print('rows:', len(uni))
    print('columns:', list(uni.columns))
    print(uni.head(3).to_string())
except NameError:
    print('\`uni\` does not exist - the fetch cell never completed.')

# 2. Did the cache file get written?
import os
print('\n=== cache file ===')
print('us_universities_coords.csv exists:', os.path.
↳exists('us_universities_coords.csv'))

# 3. What do YOUR institution names actually look like?
print('\n=== your institution names (top 10 by frequency) ===')
print(df['institution'].value_counts().head(10).to_string())

# 4. What did matching produce?
print('\n=== match results ===')
try:
    print('total institutions:', len(inst_state))
    print('matched to real coords:', inst_state['lat'].notna().sum())

```

```

    print('fell back to centroid:', inst_state['lat'].isna().sum())
    print('\nexampels that FAILED to match:')
    print(inst_state[inst_state['lat'].isna()][['institution', 'state']].
    ↪head(10).to_string())
except (NameError, KeyError) as e:
    print('inst_state not available or missing columns:', e)

# 5. If uni loaded, show a few of its normalized keys to compare against yours
print('\n=== sample of dataset names (for comparison) ===')
try:
    print(uni['uni_name'].head(10).to_string())
except (NameError, KeyError):
    print('no uni_name column to show')

```

=== uni dataframe ===

`uni` does not exist - the fetch cell never completed.

=== cache file ===

us\_universities\_coords.csv exists: False

=== your institution names (top 10 by frequency) ===

| institution                                       |     |
|---------------------------------------------------|-----|
| Carnegie Mellon University                        | 115 |
| Georgia Tech Research Corporation                 | 106 |
| University of Illinois at Urbana-Champaign        | 105 |
| Arizona State University                          | 101 |
| Regents of the University of Michigan - Ann Arbor | 101 |
| Purdue University                                 | 100 |
| University of California-San Diego                | 100 |
| University of Washington                          | 100 |
| Cornell University                                | 89  |
| University of Southern California                 | 89  |

=== match results ===

total institutions: 568

matched to real coords: 144

fell back to centroid: 424

examples that FAILED to match:

|   | institution                                                 | state |
|---|-------------------------------------------------------------|-------|
| 0 | ASSISTMENTS FOUNDATION, INC.                                | MA    |
| 1 | AUGUSTA UNIVERSITY RESEARCH INSTITUTE, INC.                 | GA    |
| 2 | Adler Planetarium                                           | IL    |
| 3 | Alabama A&M University                                      | AL    |
| 4 | Alabama State University                                    | AL    |
| 5 | Allen Institute                                             | WA    |
| 6 | American Indian Higher Education Consortium                 | VA    |
| 7 | American Institutes for Research in the Behavioral Sciences | VA    |

|   |                                        |    |
|---|----------------------------------------|----|
| 8 | American Modeling Teachers Association | CA |
| 9 | American Museum Natural History        | NY |

```
=== sample of dataset names (for comparison) ===
no uni_name column to show
```

## 1.5 5. Principal Investigator Analysis

```
[16]: pip install statsmodels
```

```
Requirement already satisfied: statsmodels in
/Users/_fin.fish_/envs/ds5500/lib/python3.9/site-packages (0.14.6)
Requirement already satisfied: numpy<3,>=1.22.3 in
/Users/_fin.fish_/envs/ds5500/lib/python3.9/site-packages (from statsmodels)
(2.0.2)
Requirement already satisfied: scipy!=1.9.2,>=1.8 in
/Users/_fin.fish_/envs/ds5500/lib/python3.9/site-packages (from statsmodels)
(1.13.1)
Requirement already satisfied: pandas!=2.1.0,>=1.4 in
/Users/_fin.fish_/envs/ds5500/lib/python3.9/site-packages (from statsmodels)
(2.3.3)
Requirement already satisfied: patsy>=0.5.6 in
/Users/_fin.fish_/envs/ds5500/lib/python3.9/site-packages (from statsmodels)
(1.0.2)
Requirement already satisfied: packaging>=21.3 in
/Users/_fin.fish_/envs/ds5500/lib/python3.9/site-packages (from statsmodels)
(26.2)
Requirement already satisfied: python-dateutil>=2.8.2 in
/Users/_fin.fish_/envs/ds5500/lib/python3.9/site-packages (from
pandas!=2.1.0,>=1.4->statsmodels) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in
/Users/_fin.fish_/envs/ds5500/lib/python3.9/site-packages (from
pandas!=2.1.0,>=1.4->statsmodels) (2026.2)
Requirement already satisfied: tzdata>=2022.7 in
/Users/_fin.fish_/envs/ds5500/lib/python3.9/site-packages (from
pandas!=2.1.0,>=1.4->statsmodels) (2026.2)
Requirement already satisfied: six>=1.5 in
/Users/_fin.fish_/envs/ds5500/lib/python3.9/site-packages (from python-
dateutil>=2.8.2->pandas!=2.1.0,>=1.4->statsmodels) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
```

```
[17]: import numpy as np
import plotly.graph_objects as go

# grand totals (denominators)
grand_awards = df['awd_id'].count()
grand_funding = df['amount'].sum()
```

```

TOP_PI = 100
TOP_PI_BUBBLE = 25

# Bar: top 100 PIs by award count
pi_total = (
    df.groupby('pi_name')['awd_id']
    .count().reset_index()
    .rename(columns={'awd_id': 'num_awards'})
    .sort_values('num_awards', ascending=False)
    .head(TOP_PI)
    .reset_index(drop=True)
)

pi_award_share = pi_total['num_awards'].sum() / grand_awards * 100
print(f'Top {TOP_PI} PIs by awards account for {pi_award_share:.1f}% of all_
↳ awards.')

fig1 = px.bar(
    pi_total, x='pi_name', y='num_awards',
    title=(f'Top {TOP_PI} Principal Investigators by Total Awards (2021-2025)'
           f'<br><sup>These PIs = {pi_award_share:.1f}% of awards</sup>'),
    labels={'num_awards': 'Total Awards', 'pi_name': 'Principal Investigator'},
    height=600
)
fig1.update_traces(marker_color='steelblue')
fig1.update_layout(xaxis_tickangle=-90, xaxis_tickfont_size=6,
                    plot_bgcolor='white', paper_bgcolor='white')
save_fig(fig1, '14_pi_awards_bar')
fig1.show()

# Bubble: top 50 PIs by award count
pi_bubble = (
    df.groupby('pi_name').agg(
        total_awards=('awd_id', 'count'),
        total_funding=('amount', 'sum')
    ).reset_index()
    .sort_values('total_awards', ascending=False)
    .head(TOP_PI_BUBBLE)
    .reset_index(drop=True)
)

pi_funding_share = pi_bubble['total_funding'].sum() / grand_funding * 100
print(f'Top {TOP_PI_BUBBLE} PIs by awards account for {pi_funding_share:.1f}%_
↳ of total funding.')

fig3 = px.scatter(
    pi_bubble, x='total_awards', y='total_funding',

```



```

size='total_awards', hover_name='pi_name',
text='pi_name',
title=(f'Top {TOP_PI_BUBBLE} Principal Investigators - Bubble Chart_
↳(2021-2025)')
    f'<br><sup>These PIs = {pi_funding_share:.1f}% of funding</sup>'),
labels={'total_funding': 'Total Funding ($)', 'total_awards': 'Number of_
↳Awards'},
color='total_funding', color_continuous_scale='Blues', height=750
)
fig3.update_traces(textposition='top center', textfont_size=7)
save_fig(fig3, '16_pi_bubble')
fig3.show()

```

Top 100 PIs by awards account for 8.1% of all awards.

Saved → plots/14\_pi\_awards\_bar.png

Top 25 PIs by awards account for 2.5% of total funding.

Saved → plots/16\_pi\_bubble.png

```

[18]: # Award Duration Distribution
df['duration_rounded'] = df['duration_years'].round(0)
avg_duration = df['duration_rounded'].dropna()

print(f'Mean award duration:    {avg_duration.mean():.2f} years')
print(f'Median award duration: {avg_duration.median():.2f} years')
print(df['duration_rounded'].value_counts().sort_index())

fig = px.histogram(
    df, x='duration_rounded',
    title='Distribution of NSF Award Durations',
    labels={'duration_rounded': 'Duration (Years)'},
    color_discrete_sequence=['steelblue'],
    nbins=7
)
fig.update_traces(xbins=dict(start=0, end=7, size=1))
fig.add_vline(x=avg_duration.mean(), line_dash='dash', line_color='red',
              annotation_text=f'Mean: {avg_duration.mean():.1f} yrs')
fig.update_layout(bargap=0.1)
save_fig(fig, '17_award_duration_hist')
fig.show()

```

Mean award duration: 2.91 years

Median award duration: 3.00 years

duration\_rounded

|     |      |
|-----|------|
| 0.0 | 170  |
| 1.0 | 683  |
| 2.0 | 1159 |
| 3.0 | 2382 |
| 4.0 | 1558 |

```
5.0      442
6.0        4
Name: count, dtype: int64
Saved → plots/17_award_duration_hist.png
```

```
[19]: # Institution → PI hierarchical analysis (top 30 schools)
DARK_COLORS = [
    '#1f3a5f', '#5c1a1a', '#1a4d2e', '#4a1f5c', '#5c3a1a',
    '#1a3a4d', '#3d1a4d', '#4d3a1a', '#1a4d4a', '#4d1a3a',
    '#2c2c54', '#5c2a1a', '#1a2e4d', '#3a4d1a', '#4d1a2a',
]

inst_pi = (
    df.groupby(['institution', 'pi_name']).agg(
        total_funding=('amount', 'sum'),
        total_awards=('awd_id', 'count')
    ).reset_index()
)

# institution totals for both metrics
inst_totals = (
    inst_pi.groupby('institution')
    .agg(inst_total=('total_funding', 'sum'),
         inst_awards=('total_awards', 'sum'))
    .reset_index()
)

# grand totals across ALL schools (denominators for the % share)
grand_funding = df['amount'].sum()
grand_awards = df['awd_id'].count()

TOP_N = 30

def school_centered_ticks(frame):
    """One tick per institution, centered under its group of bars."""
    positions, labels = [], []
    start = 0
    for inst in frame['institution'].drop_duplicates():
        n = (frame['institution'] == inst).sum()
        positions.append(start + n / 2 - 0.5)
        labels.append(inst)
        start += n
    return positions, labels

# Bar: total funding - top 30 schools by funding
```

```

top_funding = (
    inst_totals.sort_values('inst_total', ascending=False)
    .head(TOP_N).reset_index(drop=True)
)
top_funding['inst_rank'] = top_funding.index

f_share = top_funding['inst_total'].sum() / grand_funding * 100
print(f'Top {TOP_N} schools by funding account for {f_share:.1f}% of total_
    ↳funding.')

inst_pi_funding = (
    inst_pi.merge(top_funding[['institution', 'inst_rank']], on='institution')
    .sort_values(['inst_rank', 'total_funding'], ascending=[True, False])
    .reset_index(drop=True)
)
inst_pi_funding['label'] = inst_pi_funding['institution'] + ' - ' +
    ↳inst_pi_funding['pi_name']
tickvals, ticktext = school_centered_ticks(inst_pi_funding)
ycap_f = inst_pi_funding['total_funding'].quantile(0.99)

fig1 = px.bar(
    inst_pi_funding,
    x='label', y='total_funding',
    color='institution',
    title=(f'PI-Institution Pairs by Total Funding - Top {TOP_N} Schools '
        f'(2021-2025)<br><sup>These schools = {f_share:.1f}% of funding</
        ↳sup>'),
    labels={'total_funding': 'Total Funding ($)', 'label': 'Institution'},
    height=650,
    color_discrete_sequence=DARK_COLORS
)
fig1.update_layout(
    xaxis=dict(tickmode='array', tickvals=tickvals, ticktext=ticktext,
        tickangle=-90, tickfont_size=8),
    showlegend=False,
    bargap=0
)
save_fig(fig1, '31_inst_pi_funding')
fig1.show()

# Bar: total awards - top 30 schools by awards
top_awards = (
    inst_totals.sort_values('inst_awards', ascending=False)
    .head(TOP_N).reset_index(drop=True)
)
top_awards['inst_rank'] = top_awards.index

```

```

a_share = top_awards['inst_awards'].sum() / grand_awards * 100
print(f'Top {TOP_N} schools by awards account for {a_share:.1f}% of all awards.
↳')

inst_pi_awards = (
    inst_pi.merge(top_awards[['institution', 'inst_rank']], on='institution')
    .sort_values(['inst_rank', 'total_awards'], ascending=[True, False])
    .reset_index(drop=True)
)
inst_pi_awards['label'] = inst_pi_awards['institution'] + ' - ' +
↳inst_pi_awards['pi_name']
tickvals_a, ticktext_a = school_centered_ticks(inst_pi_awards)
ycap_a = inst_pi_awards['total_awards'].quantile(0.99)

fig2 = px.bar(
    inst_pi_awards,
    x='label', y='total_awards',
    color='institution',
    title=(f'PI-Institution Pairs by Total Awards - Top {TOP_N} Schools '
↳f'(2021-2025)<br><sup>These schools = {a_share:.1f}% of awards</
↳sup>'),
    labels={'total_awards': 'Number of Awards', 'label': 'Institution'},
    height=650,
    color_discrete_sequence=DARK_COLORS
)
fig2.update_layout(
    xaxis=dict(tickmode='array', tickvals=tickvals_a, ticktext=ticktext_a,
↳tickangle=-90, tickfont_size=8),
    showlegend=False,
    bargap=0
)
save_fig(fig2, '32_inst_pi_awards')
fig2.show()

```

Top 30 schools by funding account for 41.9% of total funding.

Saved → plots/31\_inst\_pi\_funding.png

Top 30 schools by awards account for 36.5% of all awards.

Saved → plots/32\_inst\_pi\_awards.png

## 1.6 6. Topic Modeling on Award Abstracts

We apply two methods: - **LDA** — classical bag-of-words probabilistic model - **BERTopic** — transformer-based with BERT embeddings + UMAP + HDBSCAN

```

[20]: # Inspect abstract column
print('=== abstract sample ===')
print(df['abstract'].iloc[0][:300])

```

```
print(f'abstract non-null: {df["abstract"].notna().sum()}')
```

```
=== abstract sample ===
```

```
particle and nuclear physics pnp are fundamentally probabilistic due to quantum
mechanics both fields rely on complex montecarlo mcbased simulators that use
random number sampling to make predictions for nearly all aspects of
experimental design and data interpretation in fact most branches of scien
abstract non-null: 6398
```

```
[21]: import re
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
from sklearn.feature_extraction.text import TfidfVectorizer

# Stop words: NLTK base + extended coverage + NSF boilerplate
stop_words_cs = set(stopwords.words('english'))
stop_words_cs.update([
    'also', 'yet', 'either', 'neither', 'often', 'usually', 'sometimes',
    'always', 'never', 'ever', 'still', 'already', 'just', 'even', 'though',
    'although', 'since', 'unless', 'whether', 'while', 'within', 'without',
    'among', 'along', 'across', 'around', 'toward', 'towards', 'upon',
    'hereby', 'herein', 'thereof', 'therein', 'whereby', 'wherein',
    'something', 'someone', 'somewhere', 'somehow', 'somewhat',
    'anything', 'anyone', 'anywhere', 'nothing', 'nobody', 'nowhere',
    'everything', 'everyone', 'everywhere', 'each', 'every', 'both',
    'several', 'few', 'less', 'least', 'much', 'more', 'most',
    'per', 'via', 'etc', 'ie', 'eg', 'thus', 'hence', 'therefore',
    'moreover', 'furthermore', 'however', 'nevertheless', 'nonetheless',
    'meanwhile', 'thereafter', 'thereby', 'therefrom',
])
stop_words_cs.update([
    'nsf', 'award', 'funded', 'funding', 'university', 'college',
    'investigator', 'researcher', 'student', 'faculty', 'professor',
    'graduate', 'undergraduate', 'phd', 'postdoc',
    'intellectual', 'merit', 'broader', 'impact',
    'mission', 'reflect', 'criterion', 'deem', 'statutory', 'worthy',
    'research', 'study', 'project', 'program', 'support', 'develop',
    'development', 'provide', 'work', 'result', 'include', 'approach',
    'framework', 'tool', 'application', 'technique', 'method', 'propose',
    'proposed', 'present', 'investigate', 'explore', 'examine', 'address',
    'enable', 'allow', 'improve', 'increase', 'reduce', 'important',
    'critical', 'novel', 'innovative', 'effective', 'efficient', 'robust',
    'scalable', 'real', 'large', 'high', 'broad', 'multiple', 'various',
    'different', 'existing', 'current', 'future', 'potential', 'significant',
    'national', 'foundation', 'science', 'scientific', 'team', 'paper',
    'publication', 'dataset', 'experiment', 'make', 'build', 'create',
```

```

'design', 'show', 'demonstrate', 'achieve', 'perform', 'apply',
'extend', 'combine', 'integrate', 'leverage', 'utilize', 'deploy',
'implement', 'system', 'model', 'data', 'information', 'knowledge',
'task', 'process', 'resource', 'environment', 'platform',
'infrastructure', 'component', 'layer', 'level', 'step', 'stage',
'aspect', 'feature', 'property', 'factor', 'element', 'type', 'class',
'category', 'group', 'number', 'case', 'example', 'instance',
'scenario', 'condition', 'outcome', 'activity', 'opportunity',
'effort', 'practice', 'enhance', 'identify', 'require', 'lead',
'share', 'access', 'plan', 'help', 'need', 'understand', 'exist',
'base', 'diverse', 'complex', 'service', 'material', 'state',
'structure', 'benefit', 'distribute', 'change', 'evaluate', 'involve',
'limit', 'range', 'institution', 'course', 'capability', 'energy',
])

# Subfield seed terms (expanded via TF-IDF against abstract corpus)
cs_subfields_seed = {
    'machine_learning': ['neural', 'deep learning', 'transformer', 'llm',
        ⇨ 'generative',
                                'reinforcement', 'supervised', 'unsupervised',
        ⇨ 'embedding',
                                'classification', 'backpropagation',
        ⇨ 'convolution', 'lstm',
                                'diffusion', 'finetuning', 'inference',
        ⇨ 'adversarial'],
    'cybersecurity': ['security', 'privacy', 'cryptography', 'malware',
        ⇨ 'cyber',
                                'encryption', 'authentication', 'intrusion',
        ⇨ 'vulnerability',
                                'firewall', 'forensic', 'breach', 'exploit',
        ⇨ 'threat',
                                'blockchain', 'trustworthy', 'identity',
        ⇨ 'authorization'],
    'networking': ['network', 'wireless', '5g', 'protocol', 'routing',
        ⇨ 'communication',
                                'bandwidth', 'latency', 'spectrum',
                                'internet', 'edge', 'cloud', 'packet', 'topology',
        ⇨ 'cellular',
                                'mimo', 'antenna', 'throughput', 'congestion',
                                'wifi', 'iot', 'sensor'],
    'robotics': ['robot', 'autonomous', 'drone', 'navigation',
        ⇨ 'manipulation',
                                'vehicle', 'planning', 'perception', 'control',
        ⇨ 'actuator',
                                'locomotion', 'swarm', 'humanoid', 'teleoperation',
                                'haptic', 'grasping', 'mapping', 'slam', 'motion'],

```

```

    'hci':
    ⇨ 'usability',
    ⇨ 'reality',
    ⇨ 'experience',
    ⇨ 'collaboration'],
    'algorithms':
    ⇨ 'streaming',

    'systems':
    ⇨ 'architecture',
    ⇨ 'runtime',
    ⇨ 'scalability',
    ⇨ 'fpga', 'cpu'],
    'software_engineering':
    ⇨ 'verification', 'devops',
    ⇨ 'documentation',
    ⇨ 'repository',

    'data_science':
    ⇨ 'bayesian',
    ⇨ 'ontology',
    ⇨ 'pattern',

    'quantum_computing':
    ⇨ 'circuit',
    ⇨ 'annealing',

    'cyberinfrastructure':
    ⇨ 'workflow',
    ⇨ 'metadata',

    ['human computer interaction', 'interface',
    'accessibility', 'visualization', 'augmented
    'virtual reality', 'wearable', 'gesture',
    'cognitive', 'crowdsourcing', 'gamification',
    'algorithm', 'complexity', 'optimization', 'graph',
    'combinatorial', 'approximation', 'randomized',
    'parallel', 'computational', 'geometry', 'sorting',
    'dynamic programming', 'heuristic', 'polynomial'],
    ['operating system', 'compiler', 'kernel', 'gpu',
    'memory', 'storage', 'cache', 'processor',
    'virtualization', 'container', 'performance',
    'fault tolerance', 'scheduling', 'concurrency',
    'software', 'testing', 'debugging',
    'specification', 'refactoring', 'maintenance',
    'agile', 'deployment', 'bug', 'patch',
    'static analysis', 'formal methods'],
    ['database', 'mining', 'analytics', 'clustering',
    'query', 'warehouse', 'knowledge graph',
    'semantic', 'retrieval', 'indexing', 'anomaly',
    'statistical', 'causal', 'inference', 'fairness'],
    ['quantum', 'qubit', 'entanglement', 'decoherence',
    'superposition', 'gate', 'error correction',
    'simulation', 'speedup', 'supremacy'],
    ['cyberinfrastructure', 'hpc', 'supercomputing',
    'reproducibility', 'cluster', 'grid', 'pipeline',
    'openscience', 'provenance', 'interoperability',
    'repository', 'portal', 'gateway', 'middleware'],

```

```

    'health_ai':          ['health', 'clinical', 'genomics',
↪'bioinformatics', 'biomedical',
                           'medical', 'patient', 'diagnosis', 'treatment',
↪'disease',
                           'imaging', 'drug', 'electronic health record',
↪'hospital',
                           'monitoring', 'epidemiology', 'precision
↪medicine'],
}

tfidf = TfidfVectorizer(max_features=5000, ngram_range=(1, 2),
↪stop_words='english')
tfidf.fit(df['abstract'].dropna())
vocab = tfidf.get_feature_names_out()

def expand_keywords(seeds, vocab):
    expanded = set(seeds)
    for term in vocab:
        if any(s in term for s in seeds):
            expanded.add(term)
    return list(expanded)

cs_subfields = {
    subfield: expand_keywords(seeds, vocab)
    for subfield, seeds in cs_subfields_seed.items()
}
cs_keywords_keep = set(w for kws in cs_subfields.values() for w in kws)

# Preprocessing
lemmatizer = WordNetLemmatizer()

def preprocess_cs(text):
    if not isinstance(text, str) or len(text) < 20:
        return []
    text = re.sub(r'<.*?>', ' ', text)
    text = re.sub(r'\d+', ' ', text)
    text = re.sub(r'[^a-zA-Z\s]', ' ', text)
    text = text.lower()
    tokens = word_tokenize(text)
    tokens = [lemmatizer.lemmatize(t, pos='v') for t in tokens]
    tokens = [lemmatizer.lemmatize(t, pos='n') for t in tokens]
    tokens = [t for t in tokens if t not in stop_words_cs and 3 < len(t) < 25]
    return tokens

def cs_keyword_ratio(tokens):
    if not tokens:
        return 0

```



```

        return sum(1 for t in tokens if t in cs_keywords_keep) / len(tokens)

# Prepare abstract dataframe only
df_text_abstract = df.copy()
df_text_abstract['tokens'] = df_text_abstract['abstract'].apply(preprocess_cs)
df_text_abstract['cs_ratio'] = df_text_abstract['tokens'].
    ↪ apply(cs_keyword_ratio)
df_text_abstract = df_text_abstract[df_text_abstract['tokens'].map(len) >= 5].
    ↪ reset_index(drop=True)
print(f'Abstract rows after filtering: {len(df_text_abstract):,}')

```

Abstract rows after filtering: 6,398

### 1.6.1 6a. LDA Topic Modeling

```

[22]: def build_gensim_corpus(tokens):
        """Build gensim dictionary and BoW corpus from a list of token lists."""
        dictionary = corpora.Dictionary(tokens)
        dictionary.filter_extremes(no_below=15, no_above=0.4)
        corpus = [dictionary.doc2bow(t) for t in tokens]
        return dictionary, corpus

def find_best_k(tokens, label, k_range=range(5, 55, 5)):
        """Search k from 5 to 50 in steps of 5 for a thorough sweep."""
        dictionary, corpus = build_gensim_corpus(tokens)
        coherence_scores = []
        for k in k_range:
            lda = LdaModel(corpus=corpus, id2word=dictionary,
                           num_topics=k, random_state=42,
                           passes=15, iterations=150,
                           alpha='auto', eta='auto')
            cm = CoherenceModel(model=lda, texts=tokens,
                                dictionary=dictionary, coherence='c_v')
            score = cm.get_coherence()
            coherence_scores.append(score)
            print(f' [{label}] k={k:2d}: coherence = {score:.4f}')
        best_k = list(k_range)[coherence_scores.index(max(coherence_scores))]
        print(f'\n [{label}] Best k = {best_k} (coherence = {max(coherence_scores):
            ↪ .4f})\n')
        return best_k, dictionary, corpus, coherence_scores, list(k_range)

print('=== Abstract ===')
best_k_abstract, dictionary_abs, corpus_abs, scores_abs, k_range_abs =
    ↪ find_best_k(
        df_text_abstract['tokens'].tolist(), 'abstract'
    )

```

```

fig_coh, ax_coh = plt.subplots(figsize=(10, 4))
ax_coh.plot(k_range_abs, scores_abs, marker='o', label='Abstract',
            color='steelblue')
ax_coh.set_title('LDA Coherence Score by Number of Topics (Abstract)')
ax_coh.set_xlabel('Number of Topics (k)')
ax_coh.set_ylabel('Coherence Score (c_v)')
ax_coh.legend()
plt.tight_layout()
save_fig(fig_coh, '18_lda_coherence')
plt.show()

print(f'Best k - Abstract: {best_k_abstract}')

```

=== Abstract ===

```

[abstract] k= 5: coherence = 0.3978
[abstract] k=10: coherence = 0.4744
[abstract] k=15: coherence = 0.4583
[abstract] k=20: coherence = 0.4804
[abstract] k=25: coherence = 0.4739
[abstract] k=30: coherence = 0.4754
[abstract] k=35: coherence = 0.4578
[abstract] k=40: coherence = 0.4504
[abstract] k=45: coherence = 0.4519
[abstract] k=50: coherence = 0.4487

```

[abstract] Best k = 20 (coherence = 0.4804)

Saved → plots/18\_lda\_coherence.png

Best k - Abstract: 20

```

[23]: # Train final LDA model using best k from coherence sweep
# best_k_abstract is set automatically from the sweep above.
# Override here if you want a fixed k: best_k_abstract = 20
print(f'Training LDA with k={best_k_abstract}')

lda_abstract = LdaModel(
    corpus=corpus_abs, id2word=dictionary_abs,
    num_topics=best_k_abstract, random_state=42,
    passes=40, iterations=400,
    alpha='auto', eta='auto',
    chunksize=2000, minimum_probability=0.01
)
print(' LDA Abstract Topics \n')
for idx, topic in lda_abstract.print_topics(num_words=10):
    words = [w.split('*')[1].replace(' ', '').strip() for w in topic.split('+')]
    print(f'Topic {idx+1:2d}: {" | ".join(words)}')

```

Training LDA with k=20

## LDA Abstract Topics

Topic 1: wireless | sense | spectrum | communication | device | patient | sensor | health | mobile | monitor  
Topic 2: security | attack | secure | vulnerability | cybersecurity | threat | detection | defense | authentication | detect  
Topic 3: privacy | event | speech | risk | machine | analysis | technology | private | individual | differential  
Topic 4: computational | simulation | engineer | device | circuit | technology | physic | physical | storage | flow  
Topic 5: decision | fairness | bias | machine | human | time | mechanism | account | fair | algorithm  
Topic 6: image | neural | deep | train | representation | object | computer | vision | network | video  
Topic 7: intelligence | artificial | agent | train | decisionmaking | healthcare | ensure | machine | aibased | domain  
Topic 8: software | code | test | language | verification | developer | formal | automate | reason | analysis  
Topic 9: conference | workshop | participant | community | travel | field | career | event | international | attend  
Topic 10: teacher | generative | biological | sequence | molecular | biology | human | emotion | health | computational  
Topic 11: community | cyberinfrastructure | scientist | workflow | software | stream | analysis | domain | open | datasets  
Topic 12: algorithm | problem | graph | theory | machine | optimization | computational | theoretical | network | goal  
Topic 13: user | social | online | medium | policy | people | interaction | content | public | behavior  
Topic 14: robot | control | autonomous | vehicle | human | safety | robotics | algorithm | realtime | physical  
Topic 15: quantum | language | communication | error | recommender | people | compression | computer | news | algorithm  
Topic 16: train | education | compute | computer | school | workforce | skill | experience | engineer | community  
Topic 17: network | device | edge | internet | smart | communication | traffic | power | performance | testbed  
Topic 18: technology | community | virtual | people | user | experience | visualization | interaction | reality | accessibility  
Topic 19: community | disaster | technology | health | resilience | city | local | public | response | mobility  
Topic 20: compute | performance | hardware | memory | architecture | software | cloud | computation | efficiency | highperformance

```
[24]: # Assign dominant topic & CS subfield labels
      # IMPORTANT: review the topic words printed above and update this dict to match.
      # Keys are 0-indexed topic numbers (0 to best_k_abstract-1).
      # The labels below match the k=20 best-k output - re-label if k changed.
```

```

def get_dominant_topic(bow, model):
    topics = model.get_document_topics(bow)
    return max(topics, key=lambda x: x[1])[0] if topics else -1

abstract_topic_labels = {
    0: 'Networking & Health Sensing',          # wireless / sense / spectrum /
    ↪patient / health / sensor
    1: 'Cybersecurity & Threat Detection',      # security / attack /
    ↪vulnerability / threat / detection
    2: 'Privacy & Fairness',                  # privacy / speech / risk /
    ↪private / differential
    3: 'Scientific Computing & Simulation',    # computational / simulation /
    ↪engineer / circuit / physics
    4: 'Fairness & Algorithmic Ethics',        # decision / fairness / bias /
    ↪fair / algorithm
    5: 'Computer Vision & Deep Learning',     # image / neural / deep /
    ↪object / vision / video
    6: 'AI Agents & Healthcare',              # intelligence / artificial /
    ↪agent / healthcare / AI-based
    7: 'Software Engineering & Formal Methods',# software / code /
    ↪verification / formal / automate
    8: 'Academic Community & Outreach',        # conference / workshop /
    ↪participant / travel / career
    9: 'Biology & Generative AI',             # teacher / generative /
    ↪biological / molecular / biology
    10: 'Cyberinfrastructure & Open Science',  # cyberinfrastructure /
    ↪workflow / datasets / open
    11: 'Algorithms, ML & Theory',            # algorithm / graph / theory /
    ↪optimization / theoretical
    12: 'Social Media & Online Behavior',      # user / social / online /
    ↪policy / content / behavior
    13: 'Robotics & Autonomous Systems',      # robot / autonomous / vehicle
    ↪/ safety / robotics
    14: 'Quantum & NLP',                     # quantum / language / error /
    ↪compression / recommender
    15: 'STEM Education & Workforce',         # train / education / school /
    ↪workforce / skill
    16: 'Networking & Edge Computing',        # network / edge / internet /
    ↪smart / traffic / testbed
    17: 'HCI & Accessibility',               # virtual / visualization /
    ↪interaction / reality / accessibility
    18: 'Resilience & Smart Cities',          # disaster / resilience / city
    ↪/ local / response / mobility
    19: 'Systems & Architecture',            # compute / hardware / memory /
    ↪architecture / cloud / HPC

```

```

}

# Pad any extra topics if best_k > 15
for i in range(len(abstract_topic_labels), best_k_abstract):
    abstract_topic_labels[i] = f'Topic {i+1}'

df_text_abstract['lda_topic'] = [
    get_dominant_topic(bow, lda_abstract) for bow in corpus_abs
]
df_text_abstract['cs_subfield'] = df_text_abstract['lda_topic'].
    ↪map(abstract_topic_labels)

print('Abstract subfield distribution:')
print(df_text_abstract['cs_subfield'].value_counts())

```

Abstract subfield distribution:

| cs_subfield                           |     |
|---------------------------------------|-----|
| STEM Education & Workforce            | 760 |
| Algorithms, ML & Theory               | 668 |
| Systems & Architecture                | 596 |
| Resilience & Smart Cities             | 454 |
| Software Engineering & Formal Methods | 428 |
| Cybersecurity & Threat Detection      | 374 |
| Networking & Health Sensing           | 367 |
| Academic Community & Outreach         | 340 |
| Robotics & Autonomous Systems         | 327 |
| Networking & Edge Computing           | 291 |
| Computer Vision & Deep Learning       | 287 |
| Social Media & Online Behavior        | 279 |
| HCI & Accessibility                   | 265 |
| Scientific Computing & Simulation     | 255 |
| Cyberinfrastructure & Open Science    | 231 |
| Fairness & Algorithmic Ethics         | 167 |
| Quantum & NLP                         | 122 |
| Biology & Generative AI               | 92  |
| Privacy & Fairness                    | 69  |
| AI Agents & Healthcare                | 26  |

Name: count, dtype: int64

```

[25]: # LDA: Awards & funding per subfield (aggregated across all years)
# With only 5 years of data, per-year trends are noisy - aggregate instead.
subfield_total = (
    df_text_abstract.groupby('cs_subfield').agg(
        num_awards=('awd_id', 'count'),
        total_funding=('amount', 'sum')
    ).reset_index()
)

```

```

for metric, xlabel, suffix in [
    ('num_awards', 'Number of Awards', 'awards'),
    ('total_funding', 'Total Funding ($)', 'funding'),
]:
    d = subfield_total.sort_values(metric, ascending=True)

    fig_bar = px.bar(
        d, x=metric, y='cs_subfield', orientation='h',
        title=f'NSF CSE {xlabel} per CS Subfield (2021-2025 total)',
        labels={metric: xlabel, 'cs_subfield': 'CS Subfield'},
        height=600,
    )
    fig_bar.update_traces(marker_color='steelblue')
    fig_bar.update_layout(margin=dict(l=240))
    save_fig(fig_bar, f'19_lda_bar_abstract_{suffix}')
    fig_bar.show()

```

Saved → plots/19\_lda\_bar\_abstract\_awards.png

Saved → plots/19\_lda\_bar\_abstract\_funding.png

```

[26]: # Total funding by LDA subfield
topic_funding = (
    df_text_abstract.groupby('cs_subfield')['amount']
    .sum().reset_index()
    .sort_values('amount', ascending=False)
)
fig = px.bar(
    topic_funding, x='amount', y='cs_subfield', orientation='h',
    title='Total NSF Funding by CS Subfield (2021-2025)',
    labels={'amount': 'Total Funding ($)', 'cs_subfield': 'CS Subfield'},
    height=600
)
fig.update_traces(marker_color='steelblue')
fig.update_layout(yaxis={'categoryorder': 'total ascending'})
save_fig(fig, '21_lda_funding_abstract')
fig.show()

```

Saved → plots/21\_lda\_funding\_abstract.png

```

[27]: # LDA topic distribution (aggregated across all years)
topic_total = (
    df_text_abstract.groupby('cs_subfield')['awd_id']
    .count().reset_index()
    .rename(columns={'awd_id': 'count'})
    .sort_values('count', ascending=True)
)
fig_bar = px.bar(

```

```

    topic_total, x='count', y='cs_subfield', orientation='h',
    title='LDA Topic Distribution',
    labels={'count': 'Number of Awards', 'cs_subfield': 'CS Subfield'},
    height=600
)
fig_bar.update_traces(marker_color='steelblue')
fig_bar.update_layout(margin=dict(l=240))
save_fig(fig_bar, '22_lda_area_abstract')
fig_bar.show()

```

Saved → plots/22\_lda\_area\_abstract.png

## 1.6.2 6b. BERTopic — Transformer-Based Topic Modeling

```

[28]: # Clean text for BERTopic (raw text, not tokens)
def clean_text(text):
    if not isinstance(text, str):
        return ''
    text = re.sub(r'<.*?>', ' ', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text

abstracts_list = df_text_abstract['abstract'].apply(clean_text).tolist()
print(f'Abstract docs: {len(abstracts_list):,}')

```

Abstract docs: 6,398

```

[29]: # BERT Embeddings
embedding_model = SentenceTransformer('all-MiniLM-L6-v2')

print('Encoding abstracts...')
embeddings_abs = embedding_model.encode(abstracts_list, show_progress_bar=True,
    ↪ batch_size=64)
print(f'Abstract embeddings shape: {embeddings_abs.shape}')

```

Encoding abstracts...

Batches: 0%| | 0/100 [00:00<?, ?it/s]

Abstract embeddings shape: (6398, 384)

```

[30]: # UMAP + HDBSCAN + BERTopic
umap_model = UMAP(n_neighbors=15, n_components=5, min_dist=0.0,
    metric='cosine', random_state=42)
hdbscan_model = HDBSCAN(min_cluster_size=30, min_samples=10,
    metric='euclidean', prediction_data=True)
vectorizer = CountVectorizer(stop_words='english', min_df=5, ngram_range=(1,
    ↪ 2))

```

```

topic_model_abstract = BERTopic(
    embedding_model=embedding_model,
    umap_model=umap_model,
    hdbscan_model=hdbscan_model,
    vectorizer_model=vectorizer,
    top_n_words=10, verbose=True
)

print('Fitting BERTopic on abstracts...')
topics_abs, probs_abs = topic_model_abstract.fit_transform(abstracts_list,
↳ embeddings_abs)
df_text_abstract['bert_topic'] = topics_abs
n_topics_abs = len(topic_model_abstract.get_topic_info()) - 1
n_outliers_abs = (df_text_abstract['bert_topic'] == -1).sum()
print(f'BERTopic (abstract): {n_topics_abs} topics, {n_outliers_abs:,} outliers_
↳ '
      f'({n_outliers_abs/len(df_text_abstract)*100:.1f}%)')

```

2026-06-16 19:57:06,523 - BERTopic - Dimensionality - Fitting the dimensionality reduction algorithm

Fitting BERTopic on abstracts...

2026-06-16 19:57:17,823 - BERTopic - Dimensionality - Completed

2026-06-16 19:57:17,823 - BERTopic - Cluster - Start clustering the reduced embeddings

2026-06-16 19:57:17,894 - BERTopic - Cluster - Completed

2026-06-16 19:57:17,896 - BERTopic - Representation - Fine-tuning topics using representation models.

2026-06-16 19:57:19,032 - BERTopic - Representation - Completed

BERTopic (abstract): 61 topics, 1,612 outliers (25.2%)

```

[31]: # BERTopic top-word bar charts
print(' Top Words per BERTopic - Abstract \n')
info = topic_model_abstract.get_topic_info()
for _, row in info[info['Topic'] != -1].head(15).iterrows():
    words = [w for w, _ in topic_model_abstract.get_topic(row['Topic'])]
    print(f"T{row['Topic']:3d} ({row['Count']:4,} docs): {' | '.join(words[:
↳ 8])}")

fig_bc = topic_model_abstract.visualize_barchart(top_n_topics=15, n_words=8)
fig_bc.update_layout(title='BERTopic Top Words - Abstract')
save_fig(fig_bc, '23_bert_barchart_abstract')
fig_bc.show()

```

Top Words per BERTopic - Abstract

T 0 ( 286 docs): wireless | spectrum | networks | network | communication |



communications | radio | 5g  
T 1 ( 243 docs): autonomous | safety | systems | learning | control | driving | vehicles | agents  
T 2 ( 214 docs): communities | community | resilience | disaster | food | water | emergency | civic  
T 3 ( 182 docs): models | medical | learning | clinical | ai | data | project | health  
T 4 ( 173 docs): online | social | media | social media | users | information | news | authentication  
T 5 ( 165 docs): memory | computing | performance | systems | hardware | parallel | project | design  
T 6 ( 149 docs): fairness | fair | algorithmic | data | causal | learning | algorithms | project  
T 7 ( 148 docs): network | internet | cloud | performance | dns | applications | project | networks  
T 8 ( 141 docs): cs | teachers | computer science | school | computer | computing | students | science  
T 9 ( 134 docs): quantum | quantum computing | computing | quantum computers | computers | problems | classical | project  
T 10 ( 132 docs): software | code | programming | verification | bug | program | tools | testing  
T 11 ( 125 docs): security | detection | attacks | attack | malware | techniques | data | cybersecurity  
T 12 ( 122 docs): security | secure | cloud | protocols | cryptographic | mpc | cryptography | project  
T 13 ( 115 docs): conference | students | doctoral | consortium | student | travel | doctoral consortium | researchers  
T 14 ( 112 docs): vr | ar | virtual | reality | immersive | virtual reality | disabilities | accessibility  
Saved → plots/23\_bert\_barchart\_abstract.png

```
[32]: # BERTopic funding by topic
bt_funding = (
    df_text_abstract[df_text_abstract['bert_topic'] != -1]
    .groupby('bert_topic')['amount']
    .sum().reset_index()
    .sort_values('amount', ascending=False)
    .head(15)
)
bt_funding['topic_label'] = bt_funding['bert_topic'].apply(
    lambda t: f"T{T}: " + topic_model_abstract.get_topic(t)[0][0]
)
fig = px.bar(
    bt_funding, x='amount', y='topic_label', orientation='h',
    title='Top 15 BERTopics by Total NSF Funding',
    labels={'amount': 'Total Funding ($)', 'topic_label': 'BERTopic'},
    height=600
)
```

```
)
fig.update_traces(marker_color='steelblue')
fig.update_layout(yaxis={'categoryorder': 'total ascending'})
save_fig(fig, '24_bert_funding_abstract')
fig.show()
```

Saved → plots/24\_bert\_funding\_abstract.png

```
[33]: # BERTopic top topics by award count
top_topics = (
    df_text_abstract[df_text_abstract['bert_topic'] != -1]['bert_topic']
    .value_counts().head(10).index.tolist()
)
topic_total = (
    df_text_abstract[df_text_abstract['bert_topic'].isin(top_topics)]
    .groupby('bert_topic')['awd_id']
    .count().reset_index()
    .rename(columns={'awd_id': 'count'})
)
topic_total['topic_label'] = topic_total['bert_topic'].apply(
    lambda t: f"T{t}: " + topic_model_abstract.get_topic(t)[0][0]
)
topic_total = topic_total.sort_values('count', ascending=True)

fig_bar = px.bar(
    topic_total, x='count', y='topic_label', orientation='h',
    title='BERTopic Top 10 Topics by Award Count',
    labels={'count': 'Number of Awards', 'topic_label': 'Topic'},
    height=550
)
fig_bar.update_traces(marker_color='steelblue')
save_fig(fig_bar, '25_bert_topics_over_time_abstract')
fig_bar.show()
```

Saved → plots/25\_bert\_topics\_over\_time\_abstract.png

```
[34]: # Topic validation: top words + top documents per LDA topic
print('=' * 70)
print('LDA TOPIC VALIDATION - Top Words vs Top Documents')
print('=' * 70)

for topic_id in range(best_k_abstract):
    # get top words for this topic
    words = [w.split('*')[1].replace('"', '').strip()
              for w in lda_abstract.print_topics(num_words=8)[topic_id][1].
              ↪split('+')]

    label = abstract_topic_labels.get(topic_id, f'Topic {topic_id+1}')
```

```

print(f'\nTopic {topic_id+1:2d} | {label}')
print(f'Top words : {" | ".join(words)}')
print('Top documents:')

# get probability of this topic for every document
topic_probs = []
for i, bow in enumerate(corpus_abs):
    doc_topics = dict(lda_abstract.get_document_topics(bow))
    prob = doc_topics.get(topic_id, 0.0)
    topic_probs.append((i, prob))

# sort by probability and take top 5
top_docs = sorted(topic_probs, key=lambda x: x[1], reverse=True)[:5]

for rank, (doc_idx, prob) in enumerate(top_docs, 1):
    title = df_text_abstract['awd_titl_txt'].iloc[doc_idx]
    abstract_snippet = df_text_abstract['abstract'].iloc[doc_idx][:150].
    ↪replace('\n', ' ')
    print(f' {rank}. [{prob:.2f}] {title}')
    print(f' {abstract_snippet}...')

print('-' * 70)

```

```

=====
LDA TOPIC VALIDATION - Top Words vs Top Documents
=====

```

Topic 1 | Networking & Health Sensing

Top words : wireless | sense | spectrum | communication | device | patient | sensor | health

Top documents:

1. [0.92] CIF: Small: Signal Processing and Learning for NOMA Millimeter-Wave Massive MIMO Systems

with the proliferation of versatile devices and dataconsuming services the quest for spectrum efficiency has led to the combination of three disruptiv...

2. [0.92] CIF: Small: Signal Processing and Learning for NOMA Millimeter-Wave Massive MIMO Systems

with the proliferation of versatile devices and dataconsuming services the quest for spectrum efficiency has led to the combination of three disruptiv...

3. [0.85] Collaborative Research: SWIFT: Nonlinear and Inseparable Radar And Data (NIRAD) Transmission Framework for Pareto Efficient Spectrum Access in Future Wireless Networks

communications and radar are two major applications of electromagnetic waves which convey information to a destination and glean information from the

...

4. [0.85] Collaborative Research: SWIFT: Nonlinear and Inseparable Radar And Data (NIRAD) Transmission Framework for Pareto Efficient Spectrum Access in Future Wireless Networks

communications and radar are two major applications of electromagnetic waves which convey information to a destination and glean information from the

...

5. [0.85] Collaborative Research: SWIFT: Nonlinear and Inseparable Radar And Data (NIRAD) Transmission Framework for Pareto Efficient Spectrum Access in Future Wireless Networks

communications and radar are two major applications of electromagnetic waves which convey information to a destination and glean information from the

...

-----

## Topic 2 | Cybersecurity & Threat Detection

Top words : security | attack | secure | vulnerability | cybersecurity | threat | detection | defense

Top documents:

1. [0.94] Collaborative Research: SaTC: CORE: Small: Towards Label Enrichment and Refinement to Harden Learning-based Security Defenses

this project aims to harden machine learning based security defenses by improving their ability to handle dynamic changes from data breaches to ransom...

2. [0.94] Collaborative Research: SaTC: CORE: Small: Towards Label Enrichment and Refinement to Harden Learning-based Security Defenses

this project aims to harden machine learning based security defenses by improving their ability to handle dynamic changes from data breaches to ransom...

3. [0.94] Collaborative Research: SaTC: CORE: Small: Towards Label Enrichment and Refinement to Harden Learning-based Security Defenses

this project aims to harden machine learning based security defenses by improving their ability to handle dynamic changes from data breaches to ransom...

4. [0.88] Collaborative Research: SaTC: CORE: Small: Securing IoT and Edge Devices under Audio Adversarial Attacks

powered by the advancement of artificial intelligence ai techniques the nextgeneration voicecontrollable iot and edge systems have substantially facil...

5. [0.88] Collaborative Research: SaTC: CORE: Small: Securing IoT and Edge Devices under Audio Adversarial Attacks

powered by the advancement of artificial intelligence ai techniques the nextgeneration voicecontrollable iot and edge systems have substantially facil...

-----

## Topic 3 | Privacy & Fairness

Top words : privacy | event | speech | risk | machine | analysis | technology | private

Top documents:

1. [0.99] Collaborative Research: CCRI: ENS: Boa 2.0: Enhancing Infrastructure for Studying Software and its Evolution at a Large Scale

in todays softwarecentric world ultralargescale software repositories eg github with hundreds of thousands of projects each are the new library of ale...

2. [0.99] Collaborative Research: CCRI: ENS: Boa 2.0: Enhancing Infrastructure for Studying Software and its Evolution at a Large Scale

in todays softwarecentric world ultralargescale software repositories eg github with hundreds of thousands of projects each are the new library of ale...

3. [0.99] Collaborative Research: CCRI: ENS: Boa 2.0: Enhancing Infrastructure for Studying Software and its Evolution at a Large Scale

in todays softwarecentric world ultralargescale software repositories eg github with hundreds of thousands of projects each are the new library of ale...

4. [0.99] Collaborative Research: CCRI: ENS: Boa 2.0: Enhancing Infrastructure for Studying Software and its Evolution at a Large Scale

in todays softwarecentric world ultralargescale software repositories eg github with hundreds of thousands of projects each are the new library of ale...

5. [0.72] Collaborative Research: SaTC: CORE: Small: Foundations for the Next Generation of Private Learning Systems

recent advances in largescale machine learning ml promise a range of benefits to society but also introduce new risks one major risk is a loss of priv...

-----  
Topic 4 | Scientific Computing & Simulation

Top words : computational | simulation | engineer | device | circuit | technology | physic | physical

Top documents:

1. [0.88] ACED: Exploring Kinetic Physics in High-Temperature Plasmas via Low-Rank Computing Techniques -- Unraveling Atomic and Nuclear Reactions

hightemperature plasmas composed of electrically charged particles interact primarily through collective effects involving many particles at once howe...

2. [0.86] ELEMENTS: Anharmonic formalism and codes to calculate thermal transport and phase change from first-principles calculations

most properties of crystalline materials can today be calculated with a high degree of accuracy via computation due to the nature of the physical theo...

3. [0.82] Collaborative Research: Elements: A task-based code for multiphysics problems in astrophysics at exascale

observing and modeling the merger of neutron stars with each other and with black holes are challenging tasks that push understanding of the universe

...

4. [0.82] Collaborative Research: Elements: A task-based code for multiphysics problems in astrophysics at exascale

observing and modeling the merger of neutron stars with each other and with black holes are challenging tasks that push understanding of the universe

...

5. [0.76] Elements: An Integrated Software Platform for Simulating Polariton Photochemical and Photophysical Processes

coupling electronic states with the quantized radiation field inside an optical cavity creates polariton states which are composed from the collective...

-----

Topic 5 | Fairness & Algorithmic Ethics

Top words : decision | fairness | bias | machine | human | time | mechanism | account

Top documents:

1. [0.77] FAI: Fairness in Machine Learning with Human in the Loop  
despite early successes and significant potential algorithmic decisionmaking systems often inherit and encode biases that exist in the training data a...
  2. [0.70] CRII: SHF: Testing Fairness in Human Decisions with Algorithmic Bias  
unfairness in human decisions has been a longstanding issue in our society that constantly threatens the equality rights of historically underrepresen...
  3. [0.68] FAI: End-To-End Fairness for Algorithm-in-the-Loop Decision Making in the Public Sector  
the goal of this project is to develop methods and tools that assist public sector organizations with fair and equitable policy interventions in areas...
  4. [0.66] CRII: CIF: Information Theoretic Measures for Fairness-aware Supervised Learning  
despite the growing success of machine learning ml systems in accomplishing complex tasks their increasing use in making or aiding consequential decis...
  5. [0.66] CRII: CIF: Information Theoretic Measures for Fairness-aware Supervised Learning  
despite the growing success of machine learning ml systems in accomplishing complex tasks their increasing use in making or aiding consequential decis...
- 

Topic 6 | Computer Vision & Deep Learning

Top words : image | neural | deep | train | representation | object | computer | vision

Top documents:

1. [0.87] Collaborative Research: RI: Medium: Learning Compositional Implicit Representations for 3D Scene Understanding  
scene understanding systems take visual inputs like images or videos and reconstruct and interpret the underlying scene in terms of 3d structure objec...
2. [0.87] Collaborative Research: RI: Medium: Learning Compositional Implicit Representations for 3D Scene Understanding  
scene understanding systems take visual inputs like images or videos and reconstruct and interpret the underlying scene in terms of 3d structure objec...

3. [0.87] Collaborative Research: RI: Medium: Learning Compositional Implicit Representations for 3D Scene Understanding

scene understanding systems take visual inputs like images or videos and reconstruct and interpret the underlying scene in terms of 3d structure objec...

4. [0.82] Collaborative Research: RI: Small: Shape Configuration Learning and Its Applications

this project aims to enhance how computers understand and recognize shapes in images and 3d data while current advanced computer systems are powerful ...

5. [0.82] Collaborative Research: RI: Small: Shape Configuration Learning and Its Applications

this project aims to enhance how computers understand and recognize shapes in images and 3d data while current advanced computer systems are powerful ...

-----

## Topic 7 | AI Agents & Healthcare

Top words : intelligence | artificial | agent | train | decisionmaking | healthcare | ensure | machine

Top documents:

1. [0.40] Collaborative Research: EducateAI: CUE-T: Designing Artificial Intelligence Curricula for All Undergrads

artificial intelligence ai is drastically changing how we approach problems and the ways we craft solutions ai awareness and competency are critical f...

2. [0.40] Collaborative Research: EducateAI: CUE-T: Designing Artificial Intelligence Curricula for All Undergrads

artificial intelligence ai is drastically changing how we approach problems and the ways we craft solutions ai awareness and competency are critical f...

3. [0.40] Collaborative Research: EducateAI: CUE-T: Designing Artificial Intelligence Curricula for All Undergrads

artificial intelligence ai is drastically changing how we approach problems and the ways we craft solutions ai awareness and competency are critical f...

4. [0.40] Collaborative Research: EducateAI: CUE-T: Designing Artificial Intelligence Curricula for All Undergrads

artificial intelligence ai is drastically changing how we approach problems and the ways we craft solutions ai awareness and competency are critical f...

5. [0.38] CRII: III: Unlearning Spatial-Temporal Imitation Learning for Trustworthy Urban Intelligence

artificial intelligence ai systems are increasingly used to understand and replicate human decisionmaking in urban environments from ridesharing and d...

-----

## Topic 8 | Software Engineering & Formal Methods

Top words : software | code | test | language | verification | developer | formal | automate

Top documents:

1. [0.92] SHF:Small: Debug Information Validation for Optimizing Compilers  
almost all modern production software is compiled with optimization  
debugging optimized code is an important part of the softwaredevelopment process  
p...
2. [0.90] Collaborative Research: SHF: Medium: Natural Language Models with Execution Data for Software Testing  
natural language processing nlp models have proven useful for various  
software engineering tasks including code completion comment generation and  
upda...
3. [0.90] Collaborative Research: SHF: Medium: Natural Language Models with Execution Data for Software Testing  
natural language processing nlp models have proven useful for various  
software engineering tasks including code completion comment generation and  
upda...
4. [0.90] CRII: SHF: An Automated and User-centered Framework for Reproducing System-level Concurrency Bugs by Analyzing Bug Reports  
reproducing a software bug is necessary to ensure that the bug exists and  
to observe its behavior it is also essential for further analysis to fix the...
5. [0.90] CRII: SHF: An Automated and User-centered Framework for Reproducing System-level Concurrency Bugs by Analyzing Bug Reports  
reproducing a software bug is necessary to ensure that the bug exists and  
to observe its behavior it is also essential for further analysis to fix the...

-----  
Topic 9 | Academic Community & Outreach

Top words : conference | workshop | participant | community | travel | field | career | event

Top documents:

1. [0.97] WORKSHOP: Doctoral Consortium at the ACM Computer-Human Interaction (CHI 2023) Conference  
this is funding to support seven promising doctoral students from the  
united states as part of a total cohort of 20 graduate students along with  
seven...
2. [0.96] Travel: NSF Student Travel Grant for the Doctoral Consortium at the ACM Conference on Human Factors in Computing Systems (CHI 2024)  
the annual computerhuman interaction chi conference has for years been  
the leading international forum for the presentation and discussion of  
research...
3. [0.95] Travel: NSF Student Travel Grant for Real World Cryptography 2024 (RWC'24)  
now in its twelfth year real world crypto is an established premier venue  
which features presentations of ideas advances and challenges from across ac...
4. [0.94] Travel: Student Support for SDM 2025 (SIAM International Conference on Data Mining)  
this project provides domestic travel support for usbased phd students to



attend the society for industrial and applied mathematics siam international...

5. [0.94] NSF Student Travel Grant for the 2022 ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD 2022)

this grant provides support for 25 usbased graduate students to participate in the 28th acm sigkdd conference on knowledge discovery and data mining k...

-----

Topic 10 | Biology & Generative AI

Top words : teacher | generative | biological | sequence | molecular | biology | human | emotion

Top documents:

1. [0.73] Collaborative Research: ACED: Building Molecule Generative Models for Drug Development via Conditional Diffusion and Multi-Property Optimization  
drug development is a critical yet notoriously resourceintensive and timeconsuming process typically taking 1015 years and costing between 1 to 16 bil...

2. [0.73] Collaborative Research: ACED: Building Molecule Generative Models for Drug Development via Conditional Diffusion and Multi-Property Optimization  
drug development is a critical yet notoriously resourceintensive and timeconsuming process typically taking 1015 years and costing between 1 to 16 bil...

3. [0.67] III:Small: Interpretable Deep Generative Models for Drug Development  
drug discovery is timeconsuming and costly it takes approximately 1015 years and between 500 million to 2 billion to fully develop a new drug molecule...

4. [0.54] Collaborative Research: RI: Small: Unified Models for Sound Quality Assessment

advances in audio processing technologies like speech enhancement audio compression and hearing aids rely on automatic methods to evaluate whether pro...

5. [0.54] Collaborative Research: RI: Small: Unified Models for Sound Quality Assessment

advances in audio processing technologies like speech enhancement audio compression and hearing aids rely on automatic methods to evaluate whether pro...

-----

Topic 11 | Cyberinfrastructure & Open Science

Top words : community | cyberinfrastructure | scientist | workflow | software | stream | analysis | domain

Top documents:

1. [0.99] Collaborative Research: Frameworks: Coupling bulk and surface processes in simulating the solid earth with ASPECT and LandLab

research in the 1960s revealed that earths outer shell is broken into a dozen or so relatively rigid plates that represent the top of a convecting sys...

2. [0.99] Collaborative Research: Frameworks: Coupling bulk and surface

processes in simulating the solid earth with ASPECT and LandLab

research in the 1960s revealed that earths outer shell is broken into a dozen or so relatively rigid plates that represent the top of a convecting sys...

3. [0.99] Collaborative Research: Frameworks: Coupling bulk and surface processes in simulating the solid earth with ASPECT and LandLab

research in the 1960s revealed that earths outer shell is broken into a dozen or so relatively rigid plates that represent the top of a convecting sys...

4. [0.99] Collaborative Research: Frameworks: Coupling bulk and surface processes in simulating the solid earth with ASPECT and LandLab

research in the 1960s revealed that earths outer shell is broken into a dozen or so relatively rigid plates that represent the top of a convecting sys...

5. [0.99] Collaborative Research: Frameworks: Coupling bulk and surface processes in simulating the solid earth with ASPECT and LandLab

research in the 1960s revealed that earths outer shell is broken into a dozen or so relatively rigid plates that represent the top of a convecting sys...

-----

Topic 12 | Algorithms, ML & Theory

Top words : algorithm | problem | graph | theory | machine | optimization | computational | theoretical

Top documents:

1. [0.90] RI: Small: Toward Maximally Efficient Sampling and Optimization for Decentralized Learning: Beyond Markovian Regime

like taking an opinion poll down sampling is a key technique for grappling with the evergrowing amounts of data used in largescale machine learning ml...

2. [0.89] AF: SMALL: Submodular Functions and Hypergraphs: Partitioning and Connectivity

submodular functions are of fundamental importance in combinatorial optimization their rich structural properties coupled with algorithmic tractabilit...

3. [0.88] AF:SMALL:RUI:UNDERSTANDING THE GEOMETRY AND DIVERSITY OF SOLUTIONS - ALGORITHMS FOR GENERATING DIVERSE SOLUTIONS IN OPTIMIZATION

classical optimization asks to select a best solution from a set of available solutions according to some criteria this project explores a new kind of...

4. [0.86] Collaborative Research: AF: Small: Foundations of Algorithms Augmented with Predictions

the ability to process and use information to make better decisions is driving breakthroughs in science and engineering and they are being materialize...

5. [0.86] Collaborative Research: AF: Small: Foundations of Algorithms Augmented with Predictions

the ability to process and use information to make better decisions is

driving breakthroughs in science and engineering and they are being materialize...

-----

#### Topic 13 | Social Media & Online Behavior

Top words : user | social | online | medium | policy | people | interaction | content

Top documents:

1. [0.91] Collaborative Research: RI: Medium: Multilingual Long-form QA with Retrieval-Augmented Language Models

this project aims to enable automatic question answering systems to produce paragraphlevel answers prior work on question answering has focused on sim...

2. [0.91] Collaborative Research: RI: Medium: Multilingual Long-form QA with Retrieval-Augmented Language Models

this project aims to enable automatic question answering systems to produce paragraphlevel answers prior work on question answering has focused on sim...

3. [0.91] Collaborative Research: RI: Medium: Multilingual Long-form QA with Retrieval-Augmented Language Models

this project aims to enable automatic question answering systems to produce paragraphlevel answers prior work on question answering has focused on sim...

4. [0.91] Collaborative Research: RI: Medium: Multilingual Long-form QA with Retrieval-Augmented Language Models

this project aims to enable automatic question answering systems to produce paragraphlevel answers prior work on question answering has focused on sim...

5. [0.89] Collaborative Research: NSF-DFG: SaTC: CORE: Small: Requirements, Metrics, and Tools for Effective User Transparency

social media users sometimes encounter harmful content such as sexual abuse that they report to the social media platform however social media platfor...

-----

#### Topic 14 | Robotics & Autonomous Systems

Top words : robot | control | autonomous | vehicle | human | safety | robotics | algorithm

Top documents:

1. [0.83] Collaborative Research: CPS: Medium: Robust to Early Termination Optimization for Safe and Reliable Control of High Performance Cyber-Physical Systems

cyberphysical systems cps for example autonomous vehicles and delivery drones consist of computational units tightly integrated with their physical en...

2. [0.83] Collaborative Research: CPS: Medium: Robust to Early Termination Optimization for Safe and Reliable Control of High Performance Cyber-Physical Systems

cyberphysical systems cps for example autonomous vehicles and delivery drones consist of computational units tightly integrated with their physical en...

3. [0.83] Collaborative Research: CPS: Medium: Robust to Early Termination Optimization for Safe and Reliable Control of High Performance Cyber-Physical Systems

cyberphysical systems cps for example autonomous vehicles and delivery drones consist of computational units tightly integrated with their physical en...

4. [0.78] Collaborative Research: CISE Crosscutting Small: CPS: AI-Powered Autonomous Systems for Dynamic In-Air Operations

this project addresses a major challenge in nextgeneration autonomous systems enabling unmanned aerial vehicles uavs to physically interact with movin...

5. [0.78] Collaborative Research: CISE Crosscutting Small: CPS: AI-Powered Autonomous Systems for Dynamic In-Air Operations

this project addresses a major challenge in nextgeneration autonomous systems enabling unmanned aerial vehicles uavs to physically interact with movin...

-----

Topic 15 | Quantum & NLP

Top words : quantum | language | communication | error | recommender | people | compression | computer

Top documents:

1. [0.85] Collaborative Research: CIF: FET: Small: Realizing Joint Detection Receivers for Quantum-enhanced Optical Communications using Photonic NISQ-era Quantum Processors

optical laser communication will constitute a key component of future spacebased data communication systems including deepspace communications it can ...

2. [0.85] Collaborative Research: CIF: FET: Small: Realizing Joint Detection Receivers for Quantum-enhanced Optical Communications using Photonic NISQ-era Quantum Processors

optical laser communication will constitute a key component of future spacebased data communication systems including deepspace communications it can ...

3. [0.85] Collaborative Research: CIF: FET: Small: Realizing Joint Detection Receivers for Quantum-enhanced Optical Communications using Photonic NISQ-era Quantum Processors

optical laser communication will constitute a key component of future spacebased data communication systems including deepspace communications it can ...

4. [0.76] Collaborative Research: OAC Core: CEAPA: A Systematic Approach to Minimize Compression Error Propagation in HPC Applications

today's highperformance computing hpc applications produce vast volumes of data for postanalysis presenting a major storage and io burden for hpc syste...

5. [0.76] Collaborative Research: OAC Core: CEAPA: A Systematic Approach to

## Minimize Compression Error Propagation in HPC Applications

today's highperformance computing hpc applications produce vast volumes of data for postanalysis presenting a major storage and io burden for hpc systems...

---

## Topic 16 | STEM Education & Workforce

Top words : train | education | compute | computer | school | workforce | skill | experience

Top documents:

1. [0.99] K12 CS Pathways for Rural and Tribal Communities  
this partnership between the national center for cs education at the college of saint scholastica hermantown community school fond du lac ojibwe school...
  2. [0.99] K12 CS Pathways for Rural and Tribal Communities  
this partnership between the national center for cs education at the college of saint scholastica hermantown community school fond du lac ojibwe school...
  3. [0.90] CUE-P: Ohio Pathways to Undergraduate Computing Success (OPUCS)  
as the demand for computing professionals in the united states continues to grow it is critical to develop programs that will increase the number of s...
  4. [0.90] CUE-P: Ohio Pathways to Undergraduate Computing Success (OPUCS)  
as the demand for computing professionals in the united states continues to grow it is critical to develop programs that will increase the number of s...
  5. [0.88] Researching Equity, Access, & Learning in Computer Science Education (REAL-CS)  
the university of oregon will expand the researching equity access learning in computer science education realcs project that supports the exploring c...
- 

## Topic 17 | Networking & Edge Computing

Top words : network | device | edge | internet | smart | communication | traffic | power

Top documents:

1. [0.88] EAGER: Collaborative Research: Towards an Extensible Internet  
the internet's basic design has remained largely unchanged since it first became commercial in the 1990s now that the internet is the centerpiece of our...
2. [0.88] EAGER: Collaborative Research: Towards an Extensible Internet  
the internet's basic design has remained largely unchanged since it first became commercial in the 1990s now that the internet is the centerpiece of our...
3. [0.88] EAGER: Collaborative Research: Towards an Extensible Internet  
the internet's basic design has remained largely unchanged since it first became commercial in the 1990s now that the internet is the centerpiece of our...
4. [0.88] EAGER: Collaborative Research: Towards an Extensible Internet  
the internet's basic design has remained largely unchanged since it first became commercial in the 1990s now that the internet is the centerpiece of our...
5. [0.77] Collaborative Research: SaTC: CORE: Medium: ONSET: Optics-enabled Network Defenses for Extreme Terabit DDoS Attacks

distributed denial of service ddos attacks continue to present a clear and imminent danger to critical network infrastructures ddos attacks have incre...

-----

#### Topic 18 | HCI & Accessibility

Top words : technology | community | virtual | people | user | experience | visualization | interaction

Top documents:

1. [0.84] Collaborative Research: HCC: Small: Robot-Rooms: Giving Form to Domestic Activity, On the Go

this projects goal is to explore how robotic systems could become deeply integrated into peoples living and working spaces via robotrooms people are i...

2. [0.84] Collaborative Research: HCC: Small: Robot-Rooms: Giving Form to Domestic Activity, On the Go

this projects goal is to explore how robotic systems could become deeply integrated into peoples living and working spaces via robotrooms people are i...

3. [0.84] Collaborative Research: HCC: Small: Robot-Rooms: Giving Form to Domestic Activity, On the Go

this projects goal is to explore how robotic systems could become deeply integrated into peoples living and working spaces via robotrooms people are i...

4. [0.78] CAREER: Advancing Remote Collaboration: Inclusive Design for People with Dementia

technology increasingly provides opportunities to interact remotely with others people with cognitive impairment can be excluded from these opportunit...

5. [0.74] Collaborative Research: CCRI: New: RFDataFactory: Principled Dataset Generation, Sharing and Maintenance Tools for the Wireless Community

applied machine learning ml research in wireless faces challenges due the inability of domain experts to easily access existing wellcurated wellstruct...

-----

#### Topic 19 | Resilience & Smart Cities

Top words : community | disaster | technology | health | resilience | city | local | public

Top documents:

1. [0.97] CIVIC-PG Track A: Sustainable Biochar to Mitigate Freshwater HABs: Building Climate Resilient Rural Communities

lakes worldwide are impacted by harmful algal blooms habs also known as cyanohabs habs can lower water quality and produce toxins impairing fisheries

...

2. [0.97] CIVIC-PG Track A: Sustainable Biochar to Mitigate Freshwater HABs: Building Climate Resilient Rural Communities

lakes worldwide are impacted by harmful algal blooms habs also known as cyanohabs habs can lower water quality and produce toxins impairing fisheries

...

3. [0.96] SCC-CIVIC-PG Track B: Improving the Post-Flood Financial Resiliency of Low- and Moderate-Income Households

this project addresses resiliency to natural disasters from a perspective

that is frequently not researched but one that has major human impact financ...

4. [0.94] SCC-CIVIC-FA Track B: Inclusive Insurance: Improving the Post-Flood Financial Resiliency of Low- and Moderate-Income Households

flooding is the costliest natural disaster and climate change is projected to escalate flood risk in many places prior research has found that low and...

5. [0.94] SCC-CIVIC-FA Track B: Inclusive Insurance: Improving the Post-Flood Financial Resiliency of Low- and Moderate-Income Households

flooding is the costliest natural disaster and climate change is projected to escalate flood risk in many places prior research has found that low and...

-----

Topic 20 | Systems & Architecture

Top words : compute | performance | hardware | memory | architecture | software | cloud | computation

Top documents:

1. [0.95] Collaborative Research: CIRC: ENS: Enabling Detailed, Open-Source Accelerator Modeling

the rapid end of moores law and dennards scaling has driven computing systems from smartphones to supercomputers to embrace heterogeneous architecture...

2. [0.95] Collaborative Research: CIRC: ENS: Enabling Detailed, Open-Source Accelerator Modeling

the rapid end of moores law and dennards scaling has driven computing systems from smartphones to supercomputers to embrace heterogeneous architecture...

3. [0.86] Collaborative Research: PPOSS: Planning: SEEr: A Scalable, Energy Efficient HPC Environment for AI-Enabled Science

aienabled science where advanced machinelearning technologies are used for surrogate models autotuning and in situ data analysis is quickly being adop...

4. [0.86] Collaborative Research: PPOSS: Planning: SEEr: A Scalable, Energy Efficient HPC Environment for AI-Enabled Science

aienabled science where advanced machinelearning technologies are used for surrogate models autotuning and in situ data analysis is quickly being adop...

5. [0.86] Collaborative Research: PPOSS: Planning: SEEr: A Scalable, Energy Efficient HPC Environment for AI-Enabled Science

aienabled science where advanced machinelearning technologies are used for surrogate models autotuning and in situ data analysis is quickly being adop...

-----

### 1.6.3 6c. Per-Division LDA Topic Modeling

The global LDA model above pools all CISE abstracts into one topic space. Here we instead fit a separate LDA model within each division (OAC, CNS, CCF, IIS), so each division's subfields

are discovered only from its own awards. This surfaces structure that the pooled model blurs together — e.g. CCF’s theory/algorithms topics vs. IIS’s ML/HCI topics.

For each division we reuse the same `preprocess_cs` tokens, run the same coherence sweep (`find_best_k`), train a final model with the division’s best `k`, and assign each award its dominant within-division topic.

```
[35]: # Per-division LDA: coherence sweep + final model for each division
# Reuses preprocess_cs tokens (already in df_text_abstract),
# build_gensim_corpus,
# and find_best_k from the global-LDA section above.

DIVISIONS = ['OAC', 'CNS', 'CCF', 'IIS']

# Smaller per-division corpora + search a tighter k range than the global sweep.
DIV_K_RANGE = range(4, 22, 2)
MIN_DOCS_PER_DIV = 50 # safety floor

div_lda = {} # div_abbr -> dict with model, dictionary, corpus, best_k,
# sub_df, topic_words

for div in DIVISIONS:
    sub = df_text_abstract[df_text_abstract['div_abbr'] == div].
    reset_index(drop=True)
    n = len(sub)
    print('=' * 70)
    print(f'Division {div}: {n:,} abstracts')
    print('=' * 70)

    if n < MIN_DOCS_PER_DIV:
        print(f' Skipping {div} - fewer than {MIN_DOCS_PER_DIV} docs.\n')
        continue

    tokens = sub['tokens'].tolist()

    # cap the sweep's max k so it never exceeds what the corpus can support
    max_k = min(20, max(4, n // 40))
    k_range = [k for k in DIV_K_RANGE if k <= max_k] or [4]

    best_k, dictionary, corpus, scores, ks = find_best_k(
        tokens, div, k_range=k_range
    )

    lda = LdaModel(
        corpus=corpus, id2word=dictionary,
        num_topics=best_k, random_state=42,
        passes=40, iterations=400,
        alpha='auto', eta='auto',
```



```

        chunksize=2000, minimum_probability=0.01,
    )

    print(f' {div} LDA topics (k={best_k}) ')
    topic_words = {}
    for idx, topic in lda.print_topics(num_words=10):
        words = [w.split('*')[1].replace(' ', '').strip() for w in topic.
        ↪split('+')]
        topic_words[idx] = words
        print(f' Topic {idx + 1:2d}: {" | ".join(words)}')
    print()

    div_lda[div] = dict(
        model=lda, dictionary=dictionary, corpus=corpus,
        best_k=best_k, sub_df=sub, topic_words=topic_words,
        scores=scores, ks=ks,
    )

print('Per-division LDA complete for:', list(div_lda.keys()))

```

```

=====
Division OAC: 1,186 abstracts
=====

```

```

[OAC] k= 4: coherence = 0.3484
[OAC] k= 6: coherence = 0.3289
[OAC] k= 8: coherence = 0.3169
[OAC] k=10: coherence = 0.3090
[OAC] k=12: coherence = 0.2947
[OAC] k=14: coherence = 0.3081
[OAC] k=16: coherence = 0.3277
[OAC] k=18: coherence = 0.2952
[OAC] k=20: coherence = 0.3216

```

```

[OAC] Best k = 4 (coherence = 0.3484)

```

```

OAC LDA topics (k=4)

```

```

Topic 1: simulation | computational | quantum | physic | dynamic | division |
earth | computer | physical | chemistry

```

```

Topic 2: performance | network | computer | algorithm | compression |
analysis | device | scale | storage | simulation

```

```

Topic 3: user | security | scientist | datasets | analysis | open | image |
domain | privacy | quality

```

```

Topic 4: network | education | workforce | technology | computational |
workshop | professional | skill | experience | campus

```

```

=====
Division CNS: 2,166 abstracts
=====

```

[CNS] k= 4: coherence = 0.3531  
[CNS] k= 6: coherence = 0.4117  
[CNS] k= 8: coherence = 0.4006  
[CNS] k=10: coherence = 0.3793  
[CNS] k=12: coherence = 0.3892  
[CNS] k=14: coherence = 0.3910  
[CNS] k=16: coherence = 0.3751  
[CNS] k=18: coherence = 0.3675  
[CNS] k=20: coherence = 0.4159

[CNS] Best k = 20 (coherence = 0.4159)

CNS LDA topics (k=20)

Topic 1: security | video | privacy | user | traffic | label | attack | private | datasets | algorithm  
Topic 2: privacy | social | user | home | account | smart | goal | medium | security | machine  
Topic 3: train | compute | cybersecurity | site | deep | education | experience | engineer | workforce | underrepresented  
Topic 4: health | quantum | visualization | user | request | prediction | population | client | software | healthcare  
Topic 5: human | behavior | language | monitor | sensor | policy | individual | medium | educational | social  
Topic 6: control | management | smart | fire | sensor | monitor | vehicle | autonomous | farm | dynamic  
Topic 7: privacy | user | authentication | content | mobile | issue | fairness | bias | decision | device  
Topic 8: cost | power | intelligence | machine | adult | older | significantly | pipeline | contribute | compute  
Topic 9: robot | software | image | test | computer | control | open | source | simulation | solution  
Topic 10: workshop | security | conference | proposal | government | discussion | focus | privacy | participant | attend  
Topic 11: food | rural | environmental | patient | live | area | time | connect | quality | measure  
Topic 12: cloud | user | construction | reality | virtual | industry | human | space | device | disability  
Topic 13: wireless | communication | optimization | edge | vehicle | performance | manufacture | realtime | autonomous | industrial  
Topic 14: software | compute | device | hardware | edge | mobile | memory | performance | cloud | architecture  
Topic 15: computer | education | school | teacher | mentor | experience | compute | pathway | partnership | field  
Topic 16: mobility | transportation | smart | city | house | public | mismatch | resident | collaboration | digital  
Topic 17: internet | security | protocol | secure | computation | performance | cryptographic | extensible | function | communication  
Topic 18: attack | privacy | security | machine | defense | vulnerability |

detection | algorithm | detect | analysis

Topic 19: disaster | resilience | response | emergency | water | civic |  
natural | city | flood | local

Topic 20: wireless | spectrum | communication | sense | testbed | algorithm |  
band | thrust | user | frequency

=====  
Division CCF: 1,445 abstracts  
=====

[CCF] k= 4: coherence = 0.4008

[CCF] k= 6: coherence = 0.3677

[CCF] k= 8: coherence = 0.3714

[CCF] k=10: coherence = 0.3958

[CCF] k=12: coherence = 0.4185

[CCF] k=14: coherence = 0.4114

[CCF] k=16: coherence = 0.4013

[CCF] k=18: coherence = 0.3919

[CCF] k=20: coherence = 0.4171

[CCF] Best k = 12 (coherence = 0.4185)

CCF LDA topics (k=12)

Topic 1: memory | symbolic | woman | nonvolatile | sense | computer |  
translation | generative | multimodal | cise

Topic 2: parallel | graph | query | cluster | analysis | matrix | computation  
| performance | workflow | domain

Topic 3: computer | teacher | school | education | community | experience |  
curriculum | teach | workshop | professional

Topic 4: problem | theory | computer | complexity | solution | many |  
computational | goal | solve | time

Topic 5: hardware | performance | memory | architecture | device | technology  
| software | train | advance | efficiency

Topic 6: network | graph | neural | analysis | problem | train | thrust |  
advance | machine | representation

Topic 7: software | privacy | legal | policy | user | accountability |  
regulation | public | security | developer

Topic 8: optimization | machine | problem | decision | algorithmic | train |  
theory | guarantee | goal | prediction

Topic 9: quantum | test | software | code | verification | language | formal  
| proof | reason | correctness

Topic 10: human | control | robot | agent | mechanism | social | safety |  
sequence | dynamic | goal

Topic 11: device | environmental | edge | stream | sustainability | center |  
carbon | source | manufacture | user

Topic 12: communication | wireless | device | image | sense | security |  
channel | power | simulation | network

Division IIS: 1,601 abstracts

=====

```
[IIS] k= 4: coherence = 0.3280
[IIS] k= 6: coherence = 0.4067
[IIS] k= 8: coherence = 0.3797
[IIS] k=10: coherence = 0.3673
[IIS] k=12: coherence = 0.3691
[IIS] k=14: coherence = 0.3486
[IIS] k=16: coherence = 0.3573
[IIS] k=18: coherence = 0.3588
[IIS] k=20: coherence = 0.3625
```

[IIS] Best k = 6 (coherence = 0.4067)

IIS LDA topics (k=6)

```
Topic 1: human | fairness | bias | user | representation | machine |
algorithm | object | image | computer
Topic 2: graph | algorithm | network | problem | neural | deep | machine |
many | domain | image
Topic 3: robot | human | technology | interaction | teacher | feedback |
people | experience | agent | education
Topic 4: health | patient | disease | brain | human | prediction |
computational | machine | clinical | algorithm
Topic 5: conference | community | workshop | field | doctoral | international
| career | consortium | travel | participant
Topic 6: language | technology | user | people | community | online | social
| speech | public | worker
```

Per-division LDA complete for: ['OAC', 'CNS', 'CCF', 'IIS']

```
[36]: # Assign dominant within-division topic to each award
# Produces df_div_topics: one row per award with its division + within-division
# topic id and the top-words string for that topic (used as a provisional
# label).

def get_dominant_topic(bow, model):
    topics = model.get_document_topics(bow)
    return max(topics, key=lambda x: x[1])[0] if topics else -1

# Optional: human-readable labels per (division, topic_id). Fill in after
# reading
# the printed top words above. Any (div, tid) not in this dict falls back to its
# top-3 words. Keys are (div_abbr, topic_id) with topic_id 0-indexed.
div_topic_labels = {
    # ('CCF', 0): 'Algorithms & Complexity',
    # ('IIS', 1): 'Computer Vision',
}
```

```

rows = []
for div, info in div_lda.items():
    model = info['model']
    sub = info['sub_df']
    tw = info['topic_words']
    for i, bow in enumerate(info['corpus']):
        tid = get_dominant_topic(bow, model)
        if tid == -1:
            label = 'Unassigned'
        else:
            label = div_topic_labels.get(
                (div, tid),
                ' / '.join(tw.get(tid, [])[:3])
            )
        rows.append({
            'awd_id':      sub['awd_id'].iloc[i],
            'year':        sub['year'].iloc[i],
            'amount':      sub['amount'].iloc[i],
            'div_abbr':    div,
            'div_topic_id': tid,
            'div_topic':   f'{div}: {label}',
        })

df_div_topics = pd.DataFrame(rows)
print(f'Assigned within-division topics to {len(df_div_topics):,} awards.')
print()
for div in div_lda:
    print(f' {div} topic distribution ')
    print(
        df_div_topics[df_div_topics['div_abbr'] == div]['div_topic']
        .value_counts()
    )
    print()

```

Assigned within-division topics to 6,398 awards.

OAC topic distribution

div\_topic

|                                           |     |
|-------------------------------------------|-----|
| OAC: network / education / workforce      | 431 |
| OAC: simulation / computational / quantum | 271 |
| OAC: performance / network / computer     | 266 |
| OAC: user / security / scientist          | 218 |

Name: count, dtype: int64

CNS topic distribution

div\_topic

|                                          |     |
|------------------------------------------|-----|
| CNS: wireless / spectrum / communication | 226 |
|------------------------------------------|-----|

|                                              |     |
|----------------------------------------------|-----|
| CNS: attack / privacy / security             | 215 |
| CNS: train / compute / cybersecurity         | 212 |
| CNS: software / compute / device             | 189 |
| CNS: disaster / resilience / response        | 154 |
| CNS: workshop / security / conference        | 141 |
| CNS: control / management / smart            | 136 |
| CNS: privacy / social / user                 | 128 |
| CNS: wireless / communication / optimization | 106 |
| CNS: cloud / user / construction             | 101 |
| CNS: internet / security / protocol          | 84  |
| CNS: computer / education / school           | 74  |
| CNS: mobility / transportation / smart       | 73  |
| CNS: privacy / user / authentication         | 65  |
| CNS: food / rural / environmental            | 52  |
| CNS: human / behavior / language             | 47  |
| CNS: robot / software / image                | 46  |
| CNS: security / video / privacy              | 45  |
| CNS: health / quantum / visualization        | 42  |
| CNS: cost / power / intelligence             | 30  |

Name: count, dtype: int64

#### CCF topic distribution

| div_topic                              |     |
|----------------------------------------|-----|
| CCF: hardware / performance / memory   | 278 |
| CCF: quantum / test / software         | 248 |
| CCF: computer / teacher / school       | 192 |
| CCF: network / graph / neural          | 144 |
| CCF: problem / theory / computer       | 131 |
| CCF: optimization / machine / problem  | 110 |
| CCF: software / privacy / legal        | 84  |
| CCF: communication / wireless / device | 84  |
| CCF: human / control / robot           | 70  |
| CCF: device / environmental / edge     | 44  |
| CCF: parallel / graph / query          | 37  |
| CCF: memory / symbolic / woman         | 23  |

Name: count, dtype: int64

#### IIS topic distribution

| div_topic                              |     |
|----------------------------------------|-----|
| IIS: graph / algorithm / network       | 393 |
| IIS: language / technology / user      | 274 |
| IIS: health / patient / disease        | 246 |
| IIS: robot / human / technology        | 239 |
| IIS: conference / community / workshop | 228 |
| IIS: human / fairness / bias           | 221 |

Name: count, dtype: int64

```

[37]: # Visualize per-division topics: funding & award count by within-division
      ↪topic
      # One subplot per division, horizontal bars sorted by total funding.

for div in div_lda:
    d = df_div_topics[df_div_topics['div_abbrev'] == div]
    agg = (
        d.groupby('div_topic')
        .agg(total_funding=('amount', 'sum'),
             num_awards=('awd_id', 'count'))
        .reset_index()
        .sort_values('total_funding', ascending=True)
    )
    # strip the "DIV: " prefix for cleaner y labels
    agg['topic_label'] = agg['div_topic'].str.replace(f'{div}: ', '',
    ↪regex=False)

    fig = make_subplots(
        rows=1, cols=2, shared_yaxes=True,
        subplot_titles=('Total Funding ($M)', 'Number of Awards'),
    )
    fig.add_trace(
        go.Bar(
            x=agg['total_funding'] / 1e6, y=agg['topic_label'],
            orientation='h', marker_color='steelblue',
            name='Funding', showlegend=False,
        ),
        row=1, col=1,
    )
    fig.add_trace(
        go.Bar(
            x=agg['num_awards'], y=agg['topic_label'],
            orientation='h', marker_color='indianred',
            name='Awards', showlegend=False,
        ),
        row=1, col=2,
    )
    full = div_names.get(div, div)
    fig.update_layout(
        title=f'{full} - Within-Division LDA Topics
    ↪(k={div_lda[div]["best_k"]})',
        height=120 + 36 * len(agg),
        margin=dict(l=220),
    )
    save_fig(fig, f'33_div_lda_{div}')
    fig.show()

```

Saved → plots/33\_div\_lda\_OAC.png

Saved → plots/33\_div\_lda\_CNS.png

Saved → plots/33\_div\_lda\_CCF.png

Saved → plots/33\_div\_lda\_IIS.png

```
[38]: # Per-division BERTopic
# Reuses embeddings_abs (already computed in the global BERTopic cell) - slices
# them per division instead of re-encoding. Fits a separate BERTopic per
# division.

DIVISIONS = ['OAC', 'CNS', 'CCF', 'IIS']
MIN_DOCS_BERT = 100 # below this, skip (too few to cluster)

# abstracts + embeddings must be row-aligned with df_text_abstract
assert len(embeddings_abs) == len(df_text_abstract), \
    'embeddings_abs is not aligned with df_text_abstract - rerun the embedding
    cell.'

div_bertopic = {} # div_abbr -> dict(model, topics, sub_index)

for div in DIVISIONS:
    mask = (df_text_abstract['div_abbr'] == div).values
    n = int(mask.sum())
    print('=' * 70)
    print(f'Division {div}: {n:,} abstracts')
    print('=' * 70)

    if n < MIN_DOCS_BERT:
        print(f' Skipping {div} - fewer than {MIN_DOCS_BERT} docs.\n')
        continue

    sub_docs = [abstracts_list[i] for i in np.where(mask)[0]]
    sub_emb = embeddings_abs[mask]

    # smaller clusters for smaller corpora than the global min_cluster_size=30
    umap_d = UMAP(n_neighbors=15, n_components=5, min_dist=0.0,
                  metric='cosine', random_state=42)
    hdb_d = HDBSCAN(min_cluster_size=15, min_samples=5,
                    metric='euclidean', prediction_data=True)
    vec_d = CountVectorizer(stop_words='english', min_df=3, ngram_range=(1, 2))

    tm = BERTopic(
        embedding_model=embedding_model,
        umap_model=umap_d, hdbscan_model=hdb_d,
        vectorizer_model=vec_d, top_n_words=10, verbose=False
    )
```



```

topics_d, _ = tm.fit_transform(sub_docs, sub_emb)

info = tm.get_topic_info()
n_top = len(info) - 1
n_out = sum(1 for t in topics_d if t == -1)
print(f' {n_top} topics, {n_out:,} outliers ({n_out/n*100:.1f}%)')
for _, row in info[info['Topic'] != -1].head(12).iterrows():
    words = [w for w, _ in tm.get_topic(row['Topic'])][:8]
    print(f"      T{row['Topic']:2d} ({row['Count']:4,}): {' | '.join(words)}")
print()

# write topic ids back, aligned to df_text_abstract rows for this division
sub_index = df_text_abstract.index[mask]
div_bertopic[div] = dict(model=tm, topics=topics_d, sub_index=sub_index)

print('Per-division BERTopic complete for:', list(div_bertopic.keys()))

# Assemble a tidy frame: one row per award with its within-division BERTopic
rows = []
for div, info in div_bertopic.items():
    tm = info['model']
    for pos, idx in enumerate(info['sub_index']):
        tid = info['topics'][pos]
        if tid == -1:
            label = f'{div}: outlier'
        else:
            top_word = tm.get_topic(tid)[0][0]
            label = f'{div}: T{tid} {top_word}'
        rows.append({
            'awd_id':      df_text_abstract.at[idx, 'awd_id'],
            'year':        df_text_abstract.at[idx, 'year'],
            'amount':       df_text_abstract.at[idx, 'amount'],
            'div_abbr':     div,
            'div_bert_id':  tid,
            'div_bert':     label,
        })

df_div_bertopic = pd.DataFrame(rows)
print(f'\nAssigned within-division BERTopics to {len(df_div_bertopic):,} awards.
↳')

```

```

=====
Division OAC: 1,186 abstracts
=====

```

```

27 topics, 119 outliers (10.0%)
  T 0 ( 95): data | project | community | earths | processes | disaster |
research | science

```

T 1 ( 84): materials | quantum | molecular | computational | chemistry | project | software | simulations

T 2 ( 81): research | network | institutions | education | cyberinfrastructure | university | regional | campus

T 3 ( 79): security | data | privacy | scientific | project | cybersecurity | research | software

T 4 ( 65): cluster | research | computing | hpc | university | support | computational | gpu

T 5 ( 58): ci | science | program | nsf | research | community | support | researchers

T 6 ( 54): data | facilities | research | scientific | science | reproducibility | project | open

T 7 ( 46): ai | nairr | ci | training | research | project | pilot | cybersecurity

T 8 ( 44): scientific | computing | workflows | network | project | data | resources | workflow

T 9 ( 42): students | research | reu | site | computing | science | reu site | undergraduate

T10 ( 39): finite element | finite | element | computational | structures | project | networks | methods

T11 ( 33): dl | deep | deep learning | applications | communication | performance | hpc | learning

=====  
Division CNS: 2,166 abstracts  
=====

47 topics, 507 outliers (23.4%)

T 0 ( 115): communities | community | resilience | water | disaster | response | emergency | civic

T 1 ( 89): memory | computing | performance | systems | applications | project | cloud | hardware

T 2 ( 85): autonomous | vehicles | safety | cps | control | systems | driving | project

T 3 ( 80): students | reu | research | site | reu site | undergraduate | science | computer

T 4 ( 77): conference | security | students | attend | travel | student | researchers | award

T 5 ( 69): mobility | transportation | community | communities | housing | transit | smart | services

T 6 ( 63): attacks | detection | learning | data | adversarial | attack | project | security

T 7 ( 63): social | online | media | social media | web | misinformation | users | accounts

T 8 ( 62): privacy | data | users | social | sharing | media | online | information

T 9 ( 51): cryptographic | mpc | protocols | security | secure | computation | project | private

T10 ( 47): edge | learning | training | data | project | video | ai |

models

T11 ( 47): aerial | uavs | unmanned | research | uav | vehicles | swarm |  
airborne

=====  
Division CCF: 1,445 abstracts  
=====

32 topics, 201 outliers (13.9%)

T 0 ( 109): graph | data | graphs | algorithms | analysis | project |  
networks | problems

T 1 ( 95): algorithms | optimization | learning | problems | project |  
fairness | machine | machine learning

T 2 ( 95): software | languages | programming | testing | program | code |  
project | tools

T 3 ( 70): learning | robots | autonomous | reinforcement learning |  
reinforcement | project | human | safety

T 4 ( 69): quantum | quantum computing | computing | problems | project |  
computers | quantum computers | quantum information

T 5 ( 60): software | legal | accountability | law | systems | software  
systems | policy | privacy

T 6 ( 57): complexity | problems | algorithms | theory | computational |  
project | algebraic | bounds

T 7 ( 55): cs | teachers | computer science | school | science | computer |  
students | teacher

T 8 ( 53): wireless | iot | communication | imaging | project | devices |  
sensing | iot devices

T 9 ( 43): hardware | performance | computing | gpu | models | memory |  
accelerators | simulation

T10 ( 38): memory | hpc | performance | systems | data | nonvolatile memory  
| nonvolatile | project

T11 ( 36): ai | ai education | education | institutions | students |  
faculty | resources | community

=====  
Division IIS: 1,601 abstracts  
=====

29 topics, 182 outliers (11.4%)

T 0 ( 214): conference | students | research | researchers | doctoral | ai |  
student | support

T 1 ( 151): models | data | project | learning | ai | health | clinical |  
healthcare

T 2 ( 123): students | learning | vr | project | teachers | virtual | design  
| support

T 3 ( 84): fairness | fair | project | algorithmic | data | machine  
learning | machine | learning

T 4 ( 66): 3d | vision | computer | project | computer vision |  
applications | visual | scene

T 5 ( 60): brain | neural | neurons | computational | activity |

```

neuroscience | circuits | project
    T 6 ( 54): data | knowledge | query | queries | project | scientific | open
data | open
    T 7 ( 52): language | models | languages | text | summarization |
multilingual | project | language processing
    T 8 ( 52): workers | work | people | research | team | project | health |
design
    T 9 ( 40): media | social media | social | news | online | information |
events | local
    T10 ( 40): ai | project | data | risk | disaster | models | management |
research
    T11 ( 38): computational | analysis | materials | protein | modeling | new
| generative | models

```

Per-division BERTopic complete for: ['OAC', 'CNS', 'CCF', 'IIS']

Assigned within-division BERTopics to 6,398 awards.

```

[39]: # Visualize per-division BERTopics: funding & award count
for div in div_bertopic:
    d = df_div_bertopic[(df_div_bertopic['div_abbrev'] == div) &
                        (df_div_bertopic['div_bert_id'] != -1)]

    if d.empty:
        print(f'{div}: no non-outlier topics to plot.')
        continue

    agg = (
        d.groupby('div_bert')
        .agg(total_funding=('amount', 'sum'), num_awards=('awd_id', 'count'))
        .reset_index()
        .sort_values('total_funding', ascending=True)
    )
    agg['topic_label'] = agg['div_bert'].str.replace(f'{div}: ', '',
    ↪regex=False)

    fig = make_subplots(rows=1, cols=2, shared_yaxes=True,
                        subplot_titles=('Total Funding ($M)', 'Number of
    ↪Awards'))
    fig.add_trace(go.Bar(x=agg['total_funding'] / 1e6, y=agg['topic_label'],
                        orientation='h', marker_color='steelblue',
    ↪showlegend=False),
                    row=1, col=1)
    fig.add_trace(go.Bar(x=agg['num_awards'], y=agg['topic_label'],
                        orientation='h', marker_color='indianred',
    ↪showlegend=False),
                    row=1, col=2)
    full = div_names.get(div, div) if 'div_names' in dir() else div

```

```

fig.update_layout(
    title=f'{full} - Within-Division BERTopics',
    height=140 + 34 * len(agg), margin=dict(l=240)
)
save_fig(fig, f'34_div_bert_{div}')
fig.show()

```

Saved → plots/34\_div\_bert\_OAC.png

Saved → plots/34\_div\_bert\_CNS.png

Saved → plots/34\_div\_bert\_CCF.png

Saved → plots/34\_div\_bert\_IIS.png

## 1.7 7. LDA vs. BERTopic Comparison

```

[40]: # Heatmap: LDA vs BERTopic assignment overlap
overlap = df_text_abstract[df_text_abstract['bert_topic'] != -1].copy()
overlap['lda_label'] = overlap['cs_subfield']
overlap['bert_label'] = 'BERT_' + overlap['bert_topic'].astype(str)

crosstab = pd.crosstab(overlap['lda_label'], overlap['bert_label']).iloc[:, :20]

fig_ht, ax_ht = plt.subplots(figsize=(18, 8))
sns.heatmap(crosstab, cmap='YlOrRd', linewidths=0.2, annot=False, ax=ax_ht)
ax_ht.set_title('LDA vs BERTopic Assignment Overlap', fontsize=14,
               fontweight='bold')
ax_ht.set_xlabel('BERTopic')
ax_ht.set_ylabel('LDA CS Subfield')
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()
save_fig(fig_ht, '27_lda_bert_heatmap_abstract')
plt.show()

```

Saved → plots/27\_lda\_bert\_heatmap\_abstract.png

```

[41]: # Subfield distribution bar chart
abs_dist = df_text_abstract['cs_subfield'].value_counts().reset_index()
abs_dist.columns = ['subfield', 'count']
abs_dist['pct'] = abs_dist['count'] / abs_dist['count'].sum() * 100

fig_dist = px.bar(
    abs_dist, x='pct', y='subfield', orientation='h',
    title='NSF CSE Abstract Subfield Distribution (% of Awards)',
    labels={'pct': '% of Awards', 'subfield': 'CS Subfield'},
    height=600
)

```

```
fig_dist.update_traces(marker_color='steelblue')
fig_dist.update_layout(yaxis={'categoryorder': 'total ascending'})
save_fig(fig_dist, '28_subfield_distribution')
fig_dist.show()
```

Saved → plots/28\_subfield\_distribution.png

```
[42]: # Top growing vs declining subfields
subfield_year = (
    df_text_abstract.groupby(['year', 'cs_subfield']).agg(
        num_awards=('awd_id', 'count'),
        total_funding=('amount', 'sum')
    ).reset_index()
)
for metric, ylabel, slug in [
    ('num_awards', '% Change in Number of Awards', 'awards'),
    ('total_funding', '% Change in Total Funding', 'funding'),
]:
    pivot = subfield_year.pivot(index='cs_subfield', columns='year',
    ↪values=metric).fillna(0)
    pivot['pct_change'] = (
        (pivot[2025] - pivot[2021]) / pivot[2021].replace(0, np.nan) * 100
    ).round(1)
    pivot = pivot.dropna(subset=['pct_change']).sort_values('pct_change',
    ↪ascending=False)
    colors = ['green' if x >= 0 else 'red' for x in pivot['pct_change']]

    fig_g, ax_g = plt.subplots(figsize=(12, 7))
    ax_g.barh(pivot.index, pivot['pct_change'], color=colors)
    ax_g.axvline(x=0, color='black', linewidth=0.8, linestyle='--')
    ax_g.set_title(f'CS Subfield Growth 2021-2025 ({ylabel})',
                  fontsize=13, fontweight='bold')
    ax_g.set_xlabel(ylabel)
    plt.tight_layout()
    save_fig(fig_g, f'29_growth_abstract_{slug}')
    plt.show()
```

Saved → plots/29\_growth\_abstract\_awards.png

Saved → plots/29\_growth\_abstract\_funding.png

```
[43]: # Heatmap: subfield x year - Total Funding ($M)
subfield_year = (
    df_text_abstract.groupby(['year', 'cs_subfield'])['amount']
    .sum().reset_index()
)
pivot = subfield_year.pivot(
    index='cs_subfield', columns='year', values='amount'
).fillna(0) / 1e6
```

```

fig_hm, ax_hm = plt.subplots(figsize=(12, 8))
sns.heatmap(pivot, cmap='YlOrRd', annot=True, fmt='.1f',
            linewidths=0.3, cbar_kws={'label': 'Funding ($M)'}, ax=ax_hm)
ax_hm.set_title('NSF CSE Funding by CS Subfield and Year ($M)',
               fontsize=13, fontweight='bold')
plt.tight_layout()
save_fig(fig_hm, '30_subfield_year_heatmap_abstract')
plt.show()

```

Saved → plots/30\_subfield\_year\_heatmap\_abstract.png

```

[44]: # Heatmap: CS Subfield x Year - Number of Awards
subfield_year = (
    df_text_abstract.groupby(['year', 'cs_subfield'])['awd_id']
    .count().reset_index()
    .rename(columns={'awd_id': 'num_awards'})
)
pivot = subfield_year.pivot(
    index='cs_subfield', columns='year', values='num_awards'
).fillna(0).astype(int)
pivot = pivot.loc[pivot.sum(axis=1).sort_values(ascending=False).index]

fig = px.imshow(
    pivot,
    text_auto=True,
    color_continuous_scale='Blues',
    aspect='auto',
    title='NSF CSE Number of Awards per CS Subfield per Year',
    labels={'x': 'Year', 'y': 'CS Subfield', 'color': 'Number of Awards'},
)
fig.update_layout(
    height=100 + 60 * len(pivot),
    xaxis=dict(tickvals=list(pivot.columns)),
    yaxis=dict(tickfont=dict(size=11)),
    coloraxis_colorbar=dict(title='Awards'),
)
fig.update_traces(textfont=dict(size=11))
save_fig(fig, '30_subfield_awards_heatmap_abstract')
fig.show()

```

Saved → plots/30\_subfield\_awards\_heatmap\_abstract.png

## 1.8 8. Save Results

```
[45]: # Save enriched CSV dataset
out_cols = ['awd_id', 'awd_titl_txt', 'year', 'amount', 'duration_years',
            'institution', 'state', 'div_abbr', 'pi_name',
            'lda_topic', 'cs_subfield', 'bert_topic']

df_text_abstract[[c for c in out_cols if c in df_text_abstract.columns]].to_csv(
    'nsf_awards_abstract_topics.csv', index=False
)
print('Saved: nsf_awards_abstract_topics.csv')
```

Saved: nsf\_awards\_abstract\_topics.csv

```
[46]: # Save LDA & BERTopic models
lda_abstract.save('lda_abstract_model')
print('LDA model saved')

topic_model_abstract.save(
    'bertopic_abstract_model', serialization='safetensors', save_ctfidf=True
)
print('BERTopic model saved')
```

LDA model saved

BERTopic model saved

## 1.9 9. Export All Plots to PDF

```
[47]: from PIL import Image
from pypdf import PdfWriter, PdfReader
import io

print(f'Plots registered: {len(SAVED_PLOTS)}')
for p in SAVED_PLOTS:
    exists = os.path.exists(p)
    print(f' {p} ← {"OK" if exists else "MISSING"}')
```

Plots registered: 53

```
plots/01_total_funding_per_year.png ← OK
plots/02_cfda_funding_trends.png ← OK
plots/04_cfda_treemap.png ← OK
plots/05_div_funding_per_year.png ← OK
plots/06_div_awards_per_year.png ← OK
plots/07_div_total_funding.png ← OK
plots/08_div_total_awards.png ← OK
plots/33_div_pgm_oac_funding.png ← OK
plots/33_div_pgm_cns_funding.png ← OK
plots/33_div_pgm_ccf_funding.png ← OK
plots/33_div_pgm_iis_funding.png ← OK
```



```

plots/33_div_pgm_oac_awards.png ← OK
plots/33_div_pgm_cns_awards.png ← OK
plots/33_div_pgm_ccf_awards.png ← OK
plots/33_div_pgm_iis_awards.png ← OK
plots/10_inst_total_bar.png ← OK
plots/11_inst_treemap.png ← OK
plots/12_inst_bubble.png ← OK
plots/13_inst_total_bar_awards.png ← OK
plots/14_inst_treemap_awards.png ← OK
plots/15_inst_bubble_awards.png ← OK
plots/13_state_map_2021.png ← OK
plots/13_state_map_2022.png ← OK
plots/13_state_map_2023.png ← OK
plots/13_state_map_2024.png ← OK
plots/13_state_map_2025.png ← OK
plots/14_pi_awards_bar.png ← OK
plots/16_pi_bubble.png ← OK
plots/17_award_duration_hist.png ← OK
plots/31_inst_pi_funding.png ← OK
plots/32_inst_pi_awards.png ← OK
plots/18_lda_coherence.png ← OK
plots/19_lda_bar_abstract_awards.png ← OK
plots/19_lda_bar_abstract_funding.png ← OK
plots/21_lda_funding_abstract.png ← OK
plots/22_lda_area_abstract.png ← OK
plots/23_bert_barchart_abstract.png ← OK
plots/24_bert_funding_abstract.png ← OK
plots/25_bert_topics_over_time_abstract.png ← OK
plots/33_div_lda_OAC.png ← OK
plots/33_div_lda_CNS.png ← OK
plots/33_div_lda_CCF.png ← OK
plots/33_div_lda_IIS.png ← OK
plots/34_div_bert_OAC.png ← OK
plots/34_div_bert_CNS.png ← OK
plots/34_div_bert_CCF.png ← OK
plots/34_div_bert_IIS.png ← OK
plots/27_lda_bert_heatmap_abstract.png ← OK
plots/28_subfield_distribution.png ← OK
plots/29_growth_abstract_awards.png ← OK
plots/29_growth_abstract_funding.png ← OK
plots/30_subfield_year_heatmap_abstract.png ← OK
plots/30_subfield_awards_heatmap_abstract.png ← OK

```

```

[48]: # Convert each PNG → single-page PDF, then merge
      # This approach is robust: no LaTeX, no nbconvert, no browser - pure Python.

```

```

PDF_OUTPUT = 'NSF_Funding_Analysis_plots.pdf'

```

```

writer      = PdfWriter()
skipped     = []

for png_path in SAVED_PLOTS:
    if not os.path.exists(png_path):
        skipped.append(png_path)
        continue
    try:
        img = Image.open(png_path).convert('RGB')
        page_buf = io.BytesIO()
        # A4 landscape at 150 dpi
        img.save(page_buf, format='PDF', resolution=150)
        page_buf.seek(0)
        reader = PdfReader(page_buf)
        for page in reader.pages:
            writer.add_page(page)
    except Exception as e:
        print(f' WARNING: could not add {png_path}: {e}')
        skipped.append(png_path)

with open(PDF_OUTPUT, 'wb') as f:
    writer.write(f)

size_mb = os.path.getsize(PDF_OUTPUT) / 1e6
print(f'\nPDF saved → {PDF_OUTPUT}  ({size_mb:.1f} MB, {len(writer.pages)}_
    ↪pages)')
if skipped:
    print(f'Skipped {len(skipped)} missing files: {skipped}')

```

PDF saved → NSF\_Funding\_Analysis\_plots.pdf (9.2 MB, 53 pages)